



o

**TRƯỜNG ĐẠI HỌC NÔNG LÂM TP HCM**  
**KHOA CÔNG NGHỆ THÔNG TIN**

**TIỂU LUẬN TỐT NGHIỆP**

**TÊN ĐỀ TÀI**

**Nghiên cứu cải tiến TickNets bằng cơ chế  
attention CBAM cho bài toán phân loại ảnh**

**Ngành : Công nghệ thông tin.**

**Niên khoá : 2022 – 2026.**

**Lớp : DH22DTC.**

**Sinh viên thực hiện : Chu Toàn Đức.**



**MẪU BÁO CÁO LUẬN VĂN, TIỂU LUẬN TỐT NGHIỆP**

**TRƯỜNG ĐẠI HỌC NÔNG LÂM TP HCM**

**KHOA CÔNG NGHỆ THÔNG TIN**

**TIỂU LUẬN TỐT NGHIỆP**

**TÊN ĐỀ TÀI**

**Nghiên cứu cải tiến TickNets bằng cơ chế**

**attention CBAM cho bài toán phân loại ảnh**

**CÁN BỘ HƯỚNG DẪN SINH VIÊN THỰC HIỆN**

**TS. Nguyễn Văn Dũ 1. Chu Toàn Đức (22130047).**

**TP.HỒ CHÍ MINH, 24 tháng 06 năm 2026**

## DANH SÁCH CHỮ VIẾT TẮT

NLP **N**atural **L**anguage **P**rocessing Xử lý ngôn ngữ tự nhiên.

CNN **C**onvolutional **N**eural **N**etwork Mạng nơ-ron tích chập.

CV **C**omputer **V**ision.

Thị giác máy tính.

DL **D**eep **L**earning Học sâu.

ANN **A**rtificial **N**eural **N**etwork Mạng nơ-ron nhân tạo.

FC **F**ully **C**onected Layer Lớp đầy đủ.

CAM **C**hannel **A**ttention **M**odule Mô-đun chú ý theo kênh.

SAM **S**patial **A**ttention **M**odule

Mô-đun chú ý theo không gian.

KNN **K**-Nearest Neighbors K láng giềng gần nhất.

## DANH MỤC HÌNH ẢNH

Hình 2.1. Minh họa về Neural Network 5

Hình 2.2 Ví dụ về kiến trúc của CNN 6

Hình 2.3 Ví dụ về Output của Convolution 8

Hình 2.4 Ví dụ về việc sử dụng hàm kích hoạt ReLU 9

Hình 2.4 Ví dụ về Max-pooling và Average-pooling 11

Hình 2.5 Minh họa Fully Connected Layer 12

Hình 2.6 Kiến trúc mô hình VGG16 13

Hình 2.7 Ví dụ cách thức hoạt động của CNN 14

Hình 2.8 Công thức tính toán output của Convolution 15

Hình 2.9 Mô hình hoạt động của CAM 17

Hình 2.10 Mô hình hoạt động của SAM 19

Hình 2.11 Mô hình CBAM 20

Hình 3.1 Ví dụ về bài toán phân loại ảnh 22

Hình 3.2 Mô hình CNN kết hợp CBAM 23

Hình 3.3 Quá trình xây dựng và huấn luyện mô hình 24

Hình 3.4 Tải tập dữ liệu 26

Hình 3.5 Mô hình CNN cho tập dữ liệu MNIST-Fashion 28

Hình 3.6 Mô hình CNN cho tập dữ liệu Cifar-10 29

Hình 3.7 Cấu trúc Confusion matrix 31

Hình 3.8 Biểu đồ loss và accuracy của mô hình CNN có áp dụng Attention và không áp dụng với tập dữ liệu MNIST-Fashion 33

Hình 3.9 Biểu đồ loss và accuracy của mô hình CNN có áp dụng Attention và không áp dụng với tập dữ liệu Cifar-10 34

## DANH MỤC BẢNG

Bảng 3.1 Bảng mô tả dữ liệu tập dữ liệu MNIST-Fashion 25

Bảng 3.2 Bảng mô tả dữ liệu tập dữ liệu Cifar-10 26

Bảng 3.3 Bảng so sánh kết quả giữa các giải thuật trên tập dữ liệu MNIST-Fashion

. 35

Bảng 3.4 Bảng so sánh kết quả giữa các giải thuật trên tập dữ liệu Cifar-10 35

## TÓM TẮT

Bài báo cáo này nghiên cứu, đề xuất và hiện thực hóa giải pháp cải tiến kiến trúc mạng nơ-ron tích chập nhẹ TickNets bằng cách tích hợp phân cấp cơ chế chú ý khối tích chập CBAM (Convolutional Block Attention Module) cho bài toán phân loại hình ảnh. Thông qua các thử nghiệm thực tiễn trên hai nhóm tập dữ liệu chuẩn học thuật Fashion-MNIST, CIFAR-10 và ứng dụng thực tế phức tạp PlantVillage, Chest-Xray-Pneumonia, kết quả thực nghiệm chỉ ra rằng giải pháp chèn CBAM phân cấp tại các điểm nối mạng không chỉ giúp loại bỏ hiệu quả nhiễu nền trên ảnh bối cảnh thực tế mà còn khắc phục tối ưu sự suy giảm hiệu năng cục bộ trên ảnh kích thước nhỏ. Dựa trên các độ đo chuẩn mực bao gồm accuracy, precision, recall và f1-score, mô hình cải tiến đề xuất mang lại hiệu suất vượt trội và ổn định hơn đáng kể so với các mô hình baseline không áp dụng hoặc chèn cơ chế chú ý sai vị trí, đóng góp một hướng đi có giá trị khoa học cao cho các ứng dụng học sâu tối giản trên thiết bị có tài nguyên hạn chế.

## MỤC LỤC

DANH SÁCH CHỮ VIẾT TẮT

DANH MỤC HÌNH ẢNH

## DANH MỤC BẢNG

## TÓM TẮT

## MỤC LỤC

## MỞ ĐẦU

1. LÝ DO CHỌN ĐỀ TÀI
2. MỤC TIÊU VÀ PHẠM VI NGHIÊN CỨU
3. Ý NGHĨA KHOA HỌC VÀ THỰC TIỄN CỦA ĐỀ TÀI

## CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

## CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

### 2.1. MẠNG NƠ-RON TÍCH CHẬP SIÊU NHẹ TICKNETS

- 2.1.1. Khái niệm và đặc điểm của họ mạng tích chập siêu nhẹ TickNets
- 2.1.2. Các phép tích chập tối giản (Depthwise Convolution và Pointwise Convolution)
- 2.1.3. Kiến trúc khối Full-Residual Point-Depth-Point (FR-PDP)
- 2.1.4. Cấu trúc xương sống co giãn kênh hình dấu tích (Tick-shape Backbone) và mô hình TickNet-small
- 2.1.5. Cơ chế lan truyền đặc trưng và cách thức hoạt động của TickNets

### 2.2. CƠ CHẾ CHÚ Ý (ATTENTION MECHANISM)

- 2.2.1. Cơ chế chú ý theo kênh Squeeze-and-Excitation (SE) mặc định trong TickNets
- 2.2.2. Mô-đun chú ý theo kênh (Channel Attention Module - CAM) của CBAM
- 2.2.3. Mô-đun chú ý theo không gian (Spatial Attention Module - SAM) của CBAM
- 2.2.4. Mô-đun chú ý khối tích chập CBAM hoạt động tuần tự (CAM + SAM)

### 2.3. PHƯƠNG PHÁP NGHIÊN CỨU

## CHƯƠNG 3: BÀI TOÁN PHÂN LOẠI HÌNH ẢNH VÀ ĐỀ XUẤT CẢI TIẾN PHÂN CẤP

- 3.1. GIỚI THIỆU BÀI TOÁN PHÂN LOẠI HÌNH ẢNH
- 3.2. ĐỀ XUẤT MÔ HÌNH TICKNETS KẾT HỢP CƠ CHẾ CHÚ Ý PHÂN CẤP (HIERARCHICAL ATTENTION)

- 3.2.1. Thiết kế kiến trúc đề xuất TickNet-small CBAM-Hook
- 3.2.2. Cơ chế chú ý phân cấp (Giữ SE nội khối FR-PDP, chèn CBAM ngoại khối tại các điểm nối Hooking points và cuối mạng)
- 3.2.3. Phân tích toán học giải quyết sự chồng chéo đặc trưng không gian giữa Depthwise Convolution và Spatial Attention Module (SAM)
- 3.3. HIỆN THỰC GIẢI PHÁP VÀ THỰC NGHIỆM
  - 3.3.1. Môi trường thực nghiệm và xây dựng mô hình
    - 3.3.1.1. Môi trường thực nghiệm (Nền tảng Kaggle, GPU T4, TensorFlow và Keras)
    - 3.3.1.2. Mô tả 4 bộ dữ liệu thực nghiệm
      - 3.3.1.2.1. Nhóm dữ liệu chuẩn học thuật: Fashion-MNIST và CIFAR-10
      - 3.3.1.2.2. Nhóm dữ liệu ứng dụng thực tiễn: PlantVillage (bệnh lá cây) và Chest-Xray-Pneumonia (viêm phổi)
    - 3.3.1.3. Các phương pháp Tiền xử lý dữ liệu và Tăng cường dữ liệu (Data Augmentation) cho từng nhóm ảnh
    - 3.3.1.4. Hiện thực hóa 3 mô hình đối chứng thực nghiệm
      - 3.3.1.4.1. Mô hình 1 (SE Baseline): TickNet-small nguyên bản sử dụng chú ý SE trong khối
      - 3.3.1.4.2. Mô hình 2 (CBAM-Local): Chèn trực tiếp CBAM thay SE cục bộ trong khối FR-PDP
      - 3.3.1.4.3. Mô hình 3 (CBAM-Hook - Đề xuất cải tiến): Chèn CBAM phân cấp tại các điểm nối và cuối mạng
    - 3.3.1.5. Cấu hình quá trình huấn luyện và tối ưu hóa
      - 3.3.1.5.1. Bộ tối ưu SGD và các hàm Loss tương ứng
      - 3.3.1.5.2. Hàm gọi lại tự động giảm tốc độ học (ReduceLROnPlateau) và dừng sớm (EarlyStopping)
  - 3.3.2. Kết quả thực nghiệm, phân tích và đánh giá
    - 3.3.2.1. Các độ đo hiệu năng chuẩn mực (Accuracy, Precision, Recall, F1-score) và cấu trúc Confusion Matrix
    - 3.3.2.2. Kết quả thực nghiệm và biểu đồ Loss/Accuracy trên tập dữ liệu chuẩn (Fashion-MNIST, CIFAR-10)
    - 3.3.2.3. Kết quả thực nghiệm và biểu đồ Loss/Accuracy trên tập dữ liệu thực tiễn (PlantVillage, Chest-Xray-Pneumonia)

3.3.2.4. Phân tích hiện tượng suy giảm hiệu năng cục bộ (CBAM-Local) và ưu thế của giải pháp đề xuất (CBAM-Hook)

3.3.2.5. Giải thích mô hình bằng trực quan hóa bản đồ nhiệt Grad-CAM (Sự tập trung vùng không gian đặc trưng thực tế)

#### CHƯƠNG 4: KẾT QUẢ, KẾT LUẬN VÀ KIẾN NGHỊ

##### 4.1. KẾT QUẢ ĐẠT ĐƯỢC CỦA ĐỀ TÀI

4.2. KẾT LUẬN (Đánh giá mức độ hoàn thành so với mục tiêu và ý nghĩa thực tiễn)

##### 4.3. KIẾN NGHỊ VÀ CÁC HƯỚNG PHÁT TRIỂN TIẾP THEO

TÀI LIỆU THAM KHẢO

#### MỞ ĐẦU

### 1. LÝ DO CHỌN ĐỀ TÀI

Phân loại hình ảnh (image classification) là một trong những bài toán cơ bản trong thị giác máy tính. Bài toán này được phát biểu như sau: cho một hình ảnh, cần xác định hình ảnh này thuộc lớp nào trong một số lớp cho trước. Ngày nay nó được áp dụng rộng rãi trong hầu hết các lĩnh vực như quản lý chất lượng sản phẩm, ô tô tự lái, y tế, ...vì vậy việc nâng cao tính chính xác của việc phân loại hình ảnh là một trong những vấn đề cấp thiết.

Trong TickNet là một kiến trúc CNN nhẹ được thiết kế nhằm giảm số lượng tham số và chi phí tính toán nhưng vẫn duy trì hiệu quả phân loại cao thông qua khối Full-Residual Point-Depth-Point (FR-PDP) và cấu trúc Tick-shape Backbone. Tuy nhiên, mô hình TickNet gốc chưa khai thác cơ chế Attention để tăng cường khả năng tập trung vào các đặc trưng quan trọng của ảnh.

Trong số các cơ chế Attention, Convolutional Block Attention Module (CBAM) là một mô-đun nhẹ, có khả năng cải thiện việc trích xuất đặc trưng bằng cách kết hợp chú ý theo kênh và chú ý theo không gian. Vì vậy, việc nghiên cứu tích hợp CBAM vào TickNet-Small nhằm nâng cao hiệu quả phân loại ảnh là một hướng nghiên cứu có ý nghĩa cả về mặt học thuật và ứng dụng thực tiễn.

Xuất phát từ những lý do trên, đề tài "Nghiên cứu cải tiến TickNet-Small bằng cơ chế Attention CBAM cho bài toán phân loại ảnh" được lựa chọn thực hiện.

## 2. MỤC TIÊU VÀ PHẠM VI NGHIÊN CỨU

Mục tiêu của đề tài hướng đến việc nghiên cứu kiến trúc TickNet-Small và cơ chế Attention CBAM, từ đó đề xuất mô hình TickNet-Small kết hợp CBAM nhằm nâng cao hiệu quả của bài toán phân loại ảnh. Đồng thời, tiến hành thực nghiệm trên nhiều bộ dữ liệu khác nhau để đánh giá khả năng cải thiện hiệu suất của mô hình so với TickNet-Small gốc.

Phạm vi nghiên cứu:

- Nghiên cứu kiến trúc TickNet, khối FR-PDP và cấu trúc Tick-shape Backbone.
- Nghiên cứu cơ chế Attention, đặc biệt là Convolutional Block Attention Module (CBAM).
- Xây dựng mô hình TickNet-Small kết hợp CBAM.
- Huấn luyện và đánh giá mô hình trên các tập dữ liệu MNIST, CIFAR-10, PlantVillage và ChestXray.
- Quá trình huấn luyện sử dụng bộ tối ưu hóa SGD kết hợp với cấu hình hàm gọi lại động bao gồm giảm tốc độ học ReduceLROnPlateau và dừng huấn luyện sớm EarlyStopping.
- Đánh giá hiệu năng dựa trên các độ đo tiêu chuẩn: Accuracy, Precision, Recall, F1-score cùng trực quan hóa bằng Confusion Matrix và bản đồ nhiệt Grad-CAM để giải thích vùng chú ý của mô hình.

## 3. Ý NGHĨA KHOA HỌC VÀ THỰC TIỄN CỦA ĐỀ TÀI

Ý nghĩa khoa học:

- Giải quyết xung đột kiến trúc trong mạng siêu nhẹ: Đề tài đóng góp một lời giải khoa học sâu sắc cho việc tối ưu hóa hiệu năng các mạng nơ-ron siêu nhẹ bằng cách chỉ ra và xử lý thành công sự chồng chéo thông tin giữa phép tích chập chiều sâu và mô-đun chú ý không gian trong các cấu trúc Perceptron cục bộ.
- Đề xuất mô hình chú ý phân cấp mới: Nghiên cứu tiên phong trong việc thiết lập cấu trúc phối hợp hai tầng cơ chế chú ý: giữ nguyên attention kênh SE ở cấp độ khối cơ bản để lọc đặc trưng cục bộ, và áp dụng attention hỗn hợp CBAM ở cấp độ điểm nối để kiểm soát đặc trưng toàn cục. Điều này mở ra một hướng tiếp cận mới mẻ trong việc thiết kế và tích hợp các khối attention vào các mạng nơ-ron tối giản.
- Góp phần phát triển lý thuyết học sâu giải thích được : Thông qua việc ứng dụng thuật toán Grad-CAM để trực quan hóa các bản đồ nhiệt chú ý, đề tài cung cấp các minh chứng khoa học trực quan, rõ ràng giải thích



tại sao mô hình cải tiến có thể hội tụ nhanh hơn, định vị chính xác hơn các vùng bệnh lý hoặc vật thể so với mô hình baseline thông thường...

Ý nghĩa thực tiễn: kết quả nghiên cứu trình bày trong tiểu luận có ý nghĩa rất lớn khi áp dụng vào các bài toán trong thực tiễn như sau

- Nâng cao chất lượng chẩn đoán y khoa và quản lý nông nghiệp: Kết quả thực nghiệm của mô hình cải tiến trên tập dữ liệu Chest-Xray-Pneumonia giúp tối ưu hóa độ chính xác và giảm thiểu tỷ lệ bỏ sót trong việc chẩn đoán viêm phổi qua hình ảnh X-quang phổi. Đồng thời, thực nghiệm trên tập PlantVillage giúp hỗ trợ bà con nông dân nhận diện sớm và chính xác các loại bệnh trên lá cây trồng, góp phần thúc đẩy các giải pháp nông nghiệp thông minh, công nghệ cao.
- Tính khả thi cao trong triển khai thực tế trên thiết bị biên: Nhờ kế thừa ưu điểm siêu nhẹ của mạng TickNets, mô hình TickNet-small cải tiến bằng CBAM phân cấp cực kỳ phù hợp để đóng gói và vận hành trực tiếp trên các thiết bị di động cá nhân, máy tính bảng cầm tay, thiết bị bay không người lái hay các chip nhúng biên IoT có năng lượng và năng lực tính toán hạn chế mà không cần phụ thuộc vào máy chủ đám mây.
- 

## CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

### 1.1. Bối cảnh lĩnh vực phân loại hình ảnh và sự phát triển của học sâu (Deep Learning)

*[Nội dung phần này đang được cập nhật/viết tiếp...]*

### 1.2. Thách thức về tài nguyên tính toán và sự cần thiết của mạng nơ-ron siêu nhẹ (Light-weight CNNs)

Mặc dù các nghiên cứu tích hợp cơ chế chú ý như CBAM đã chứng minh hiệu quả vượt trội khi kết hợp với các mạng tích chập truyền thống như VGG16, việc áp dụng cơ chế này lên các kiến trúc mạng siêu nhẹ vẫn đối mặt với nhiều thách thức lớn về sự tương thích cấu trúc. Các mạng siêu nhẹ phổ biến hiện nay như MobileNets hay ShuffleNets thường chỉ sử dụng cơ chế chú ý một hướng SE để tái cấu trúc kênh đặc trưng nhằm tiết kiệm tham số. Kiến trúc mạng siêu nhẹ cải tiến gần đây là TickNets – nổi bật với khối FR-PDP và cấu trúc xương sống co giãn kênh hình dấu tích tuy đạt hiệu suất cao nhưng cũng mới chỉ dừng lại ở việc tích hợp mặc định cơ chế attention SE ở mức cục bộ bên trong khối.

### 1.3. Các nghiên cứu liên quan về cơ chế chú ý (Attention Mechanisms) trong thị giác máy tính

Cơ chế chú ý (Attention Mechanism) là một trong những cơ chế được sử dụng phổ biến trong các bài toán xử lý ngôn ngữ tự nhiên (NLP), đặc biệt là trong các mô hình dịch máy. Một trong những thành tựu nổi bật của nó là sự phát triển của mô hình Transformer, giới thiệu trong bài báo “Attention is All You Need” của Vaswani et al. năm 2017 [1]. Mô hình Transformer đã thay đổi cách tiếp cận dịch máy và các bài toán xử lý ngôn ngữ tự nhiên (NLP) nhờ vào cơ chế Attention mạnh mẽ của nó. Góp phần dẫn đến sự phát triển của các mô hình như BERT (Bidirectional Encoder Representations from Transformers) và GPT (Generative Pre-trained Transformer). Những mô hình này đã thiết lập các chuẩn mực mới trong nhiều bài toán NLP khác nhau, bao gồm dịch máy, hiểu ngôn ngữ tự nhiên, và tạo văn bản.

### 1.4. Phát biểu bài toán, giới hạn nghiên cứu và giải pháp đề xuất cải tiến của tiểu luận

Do đó, bài nghiên cứu này tập trung vào giải pháp cải tiến mạng TickNet-small bằng cơ chế chú ý phân cấp. Phương pháp này đề xuất giữ nguyên SE cục bộ trong khối FR-PDP, đồng thời chèn CBAM một cách chiến lược tại các điểm nối mạng giữa hai backbone co giãn và trước lớp Global Average Pooling cuối mạng, giúp mô hình phát huy tối đa sức mạnh lọc nhiễu không gian toàn cục mà không làm bùng nổ tham số tính toán.

## CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

### 2.1. MẠNG NƠ-RON TÍCH CHẬP SIÊU NHẹ TICK-NETS:

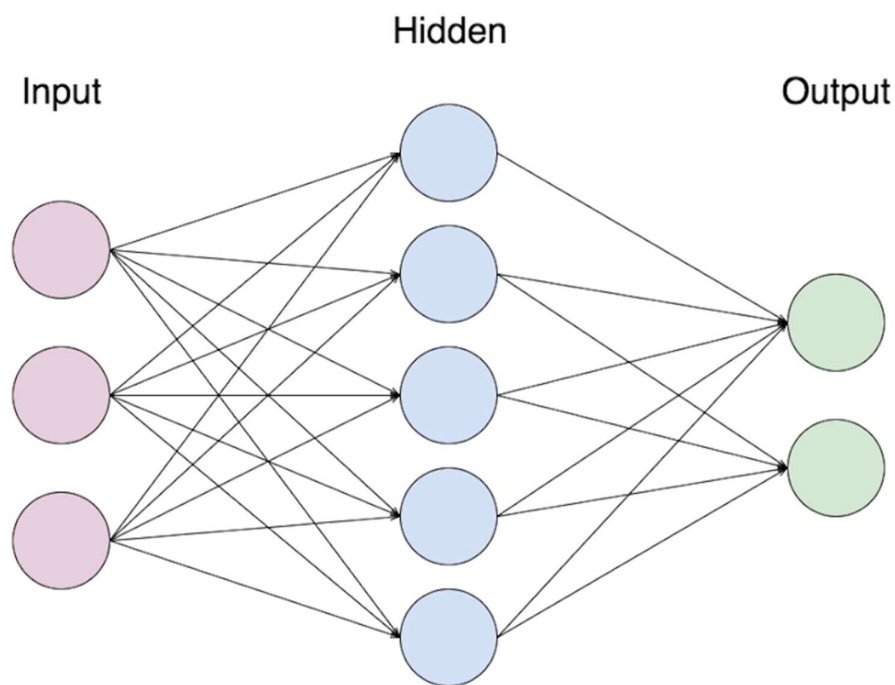
#### 2.1.1. Khái niệm và đặc điểm của họ mạng tích chập siêu nhẹ TickNets

Neural Network được mô phỏng theo bộ não của con người. Có các nút riêng lẻ tạo thành các lớp trong mạng, giống như các nơ-ron trong não của chúng ta kết nối các khu vực khác nhau. Đầu vào cho các nút trong một lớp sẽ có trọng số được gán cho chúng để thay đổi ảnh hưởng của tham số đó đến kết quả dự đoán tổng thể. Vì các trọng số được ấn định trên các liên kết giữa các nút nên mỗi nút có thể bị ảnh hưởng bởi nhiều trọng số. Mạng nơ-ron lấy tất cả dữ liệu huấn luyện ở lớp đầu vào. Sau đó, nó chuyển dữ liệu qua các lớp ẩn, chuyển đổi các giá trị dựa trên trọng số tại mỗi nút. Cuối cùng nó trả về một giá trị ở lớp đầu ra. [2]

### Kiến trúc mạng nơ-ron:

Một Neural Network thường bao gồm 3 tầng cụ thể như sau:

- **Input Layer (Tầng đầu vào):** giá trị các node chính là số lượng đặc trưng của dữ liệu đầu vào khi đưa vào mô hình. Chúng ta thấy giá trị đầu vào là  $n$  thuộc tính/đặc trưng.
- **Hidden Layer (Tầng ẩn):** có số node ẩn thường không được xác định, thường do kinh nghiệm của người thiết kế hoặc qua quá trình thử nghiệm nhiều lần mà có được. Tuy nhiên thực tế nếu số lượng node ẩn quá nhiều thì mạng sẽ học chậm, còn nếu số node quá ít thì mạng sẽ không rút trích đủ các thông tin cần thiết trên các đặc trưng. Từ đó hiệu quả của mô hình sẽ không được chính xác. Số lượng các lớp ẩn ở đây có thể một hoặc nhiều lớp ẩn tùy thuộc vào tính chất cũng như độ phức tạp của dữ liệu.
- **Output Layer (Tầng đầu ra):** giá trị các node chính là số lượng nhãn đầu ra mà chúng ta mong muốn. Ví dụ như trong tập dữ liệu của chúng ta có tổng cộng 5 nhãn, thì đầu ra của chúng ta tại lớp này chính là một lớp ẩn với 5 phân tử tương ứng với 5 nhãn.



Hình 2.1. Minh họa về Neural Network

### Khái niệm mạng nơ-ron tích chập (CNN):

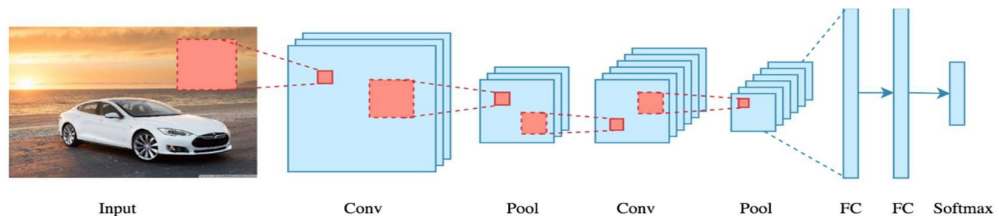
Mạng nơ-ron tích chập là một loại mạng nơ-ron đặc biệt có nhiều lớp. CNN xử lý dữ liệu có sự sắp xếp giống như lưới, sau đó trích xuất các tính năng quan trọng. CNN được xây dựng để tận dụng các mối quan hệ không gian (spatial relationships) trong dữ liệu đầu vào, làm cho chúng đặc biệt hiệu quả trong các tác vụ liên quan đến thị giác máy tính (Computer Vision.) Một lợi thế rất lớn của việc sử dụng CNN là bạn không cần thực hiện nhiều bước xử lý trước đối với hình ảnh.

CNN là một trong những mô hình học sâu phổ biến nhất và được sử dụng phổ biến trong cộng đồng thị giác máy tính (CV). CNN được dùng trong nhiều bài toán như phân loại ảnh, phân tích video, ảnh MRI, hoặc cho các bài toán của lĩnh vực xử lý ngôn ngữ tự nhiên, và hầu hết đều cho ra kết quả tốt hơn so với các mô hình khác.

Mạng CNN xử lý dữ liệu đầu vào trong không gian ba chiều: chiều rộng, chiều cao và chiều sâu. Các Nơ-ron trong mạng không liên kết hoàn toàn với toàn bộ Nơ-ron kế đến mà chỉ liên kết tới một vùng nhỏ. Cuối cùng, một tầng đầu ra được tối giản thành vec-tơ của giá trị xác suất, đại diện cho kết quả dự đoán.

### Kiến trúc mạng CNN:

Một mạng CNN bao gồm tập hợp các tầng cơ bản: convolution layer, pooling layer và fully connected layer. Ở giữa các tầng convolution và pooling thường có các hàm kích hoạt phi tuyến (nonliner layer) thường là ReLU hoặc Tanh. Ảnh khi đưa vào mạng sẽ được lan truyền qua tầng convolution layer, giá trị tính được từ các tầng convolution sẽ đi qua một hàm kích hoạt, sau đó giá trị này sẽ được lan truyền qua pooling layer, quá trình này có thể lặp đi lặp lại nhiều lần trước khi đến tầng fully connected layer. Cuối cùng ảnh sẽ được lan truyền đến tầng fully connected layer và đi qua hàm kích hoạt Softmax, thường thì cuối cùng sẽ thu được một vector chứa xác suất phần trăm thuộc về các lớp đối với các bài toán phân loại.



Hình 2.2 Ví dụ về kiến trúc của CNN

### Convolution layer:

Convolution layer là tầng quan trọng nhất và cũng là tầng đầu tiên của của mô hình CNN. Tầng này có chức năng chính là phát hiện các đặc trưng có tính không gian hiệu quả. Trong tầng này có 4 đối tượng chính là: ma trận đầu vào, bộ filters, và receptive field, feature map. Convolution layer nhận đầu vào là một ma trận 3 chiều và một bộ filters cần phải học. Bộ filters này sẽ trượt qua từng vị trí trên bức ảnh để tính tích chập (convolution) giữa bộ filter và phần tương ứng trên bức ảnh. Phần tương ứng này trên bức ảnh gọi là receptive field, tức là vùng mà một neuron có thể nhìn thấy để đưa ra quyết định, và ma trận cho ra bởi quá trình này được gọi là feature map.

Khi đưa ảnh vào mạng, bộ filter sẽ quét qua toàn bộ ảnh cho nên các đặc trưng cơ bản của ảnh như là góc, cạnh, màu sắc và texture sẽ được mạng phát hiện ra bất kể nó nằm ở vị trí nào trong ảnh. Do đó tầng convolution được xem như là một bộ trích chọn đặc trưng (feature detector) vì nó có chức năng chính là phát hiện đặc trưng cụ thể của bức ảnh đầu vào. Khi áp dụng phép tính tích chập cho xử lý hình ảnh, người ta nhận thấy rằng kỹ thuật tích chập này sẽ giúp biến đổi các thông tin đầu vào thành các yếu tố đặc trưng (nó tương tự như bộ phát hiện nhằm phát hiện ra các đặc trưng như cạnh, hướng, ...). Cụ thể hơn, tích chập sẽ trích xuất đặc trưng của ảnh đầu vào qua các vùng ảnh nhỏ. Các vùng này được gọi là Local Receptive Field (LRF). Tích chập sẽ tính toán trên các LRF chồng lấp lên nhau. Độ chồng lấp này phụ thuộc vào hệ số trượt  $S$  (stride) của từng kiến trúc mạng cụ thể. Nếu sử dụng với hệ số trượt  $S = 1$ , thì tương ứng LRF (bằng kích thước với kernel) sẽ dịch chuyển đơn vị pixel sau mỗi lần tích chập. [3]

**Padding:** là kỹ thuật thêm các pixel xung quanh biên của đầu vào (thường là các giá trị 0) trước khi áp dụng phép tích chập. Mục đích của padding là điều chỉnh kích thước của đầu vào để đạt được các mục đích cụ thể. (Ví dụ với đầu vào có kích thước  $7 \times 7$  với bộ filter có kích thước  $3 \times 3$  và stride bằng 2 thì kích thước để bộ filter trượt với stride bằng 2 là không phù hợp từ đó sử dụng padding để chèn thêm cho ở viền của đầu vào những phần tử 0 để bộ filter có thể trượt với kích thước phù hợp).

**Stride:** là số bước di chuyển của bộ filter qua ảnh đầu vào khi thực hiện phép tích chập. Nó xác định khoảng cách giữa hai vùng lân cận được xử lý liên tiếp. Stride ảnh hưởng đến kích thước của đầu ra sau mỗi lớp.

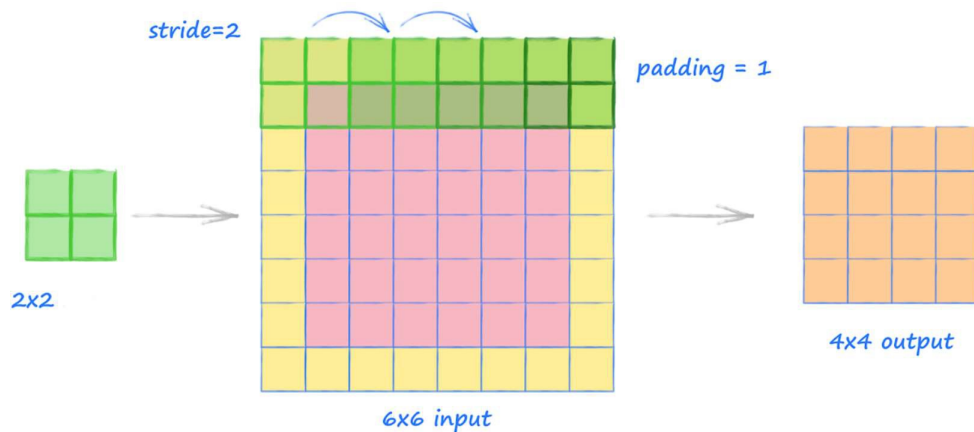
**Bias:** là một giá trị được cộng vào sau khi thực hiện phép tích chập. Nó giúp dịch chuyển (shift) giá trị đầu ra, cho phép mô hình học các đặc trưng tốt hơn và linh hoạt hơn. Mỗi bộ lọc có một bias riêng, và nó được học trong quá trình huấn luyện cùng với các trọng số của bộ lọc.

**Shared Weights:** Trong một lớp tích chập, các bộ lọc (filter) sẽ có cùng một tập trọng số được áp dụng cho toàn bộ đầu vào. Điều này có nghĩa là cùng một bộ lọc sẽ trượt qua toàn bộ đầu vào để tạo ra các bản đồ đặc trưng (feature map). Nhờ vậy, số

lượng tham số cần huấn luyện sẽ giảm đáng kể, giúp mô hình dễ dàng tránh được hiện tượng overfitting. (Ví dụ chúng ta có một hình ảnh đầu vào 5x5 và một bộ lọc 3x3. Nếu không có chia sẻ trọng số, mỗi vị trí trên hình ảnh cần một bộ trọng số riêng, điều này sẽ tạo ra số lượng tham số rất lớn. Với trọng số chia sẻ, bộ lọc 3x3 sẽ sử dụng cùng một tập trọng số để quét qua toàn bộ hình ảnh, chỉ cần huấn luyện 9 trọng số (cho bộ lọc 3x3) và 1 giá trị bias).

**Shared Bias:** Tương tự như trọng số chia sẻ, các giá trị bias cũng được chia sẻ cho toàn bộ đầu vào khi bộ lọc di chuyển. Mỗi bộ lọc sẽ có một giá trị bias duy nhất, áp dụng cho tất cả các vị trí mà bộ lọc này hoạt động.

Ví dụ có một ảnh đầu vào có kích thước 6x6 với stride bằng 2, padding bằng 1 và bộ filter có kích thước 3x3 thì đầu ra sẽ là 4x4 như hình bên dưới:



Hình 2.3 Ví dụ về Output của Convolution

Theo đó ta có thể tính kích thước đầu ra thông qua công thức sau:

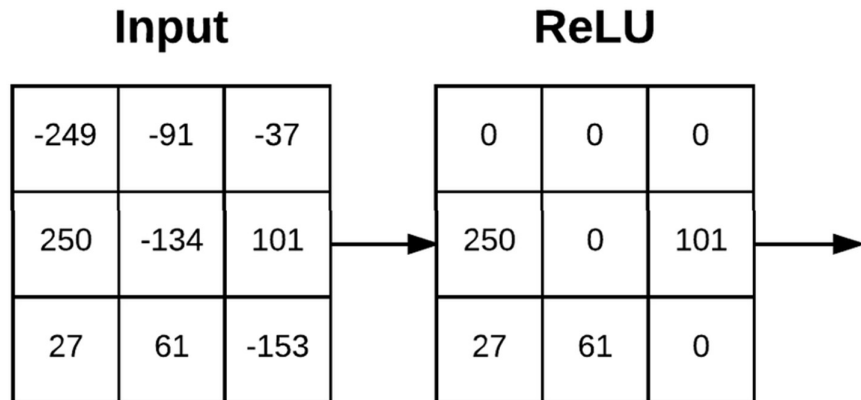
$$\text{output size} = \frac{(\text{input size}) + 2 * \text{padding} - (\text{filter size} - 1) * \text{stride}}{\text{stride}} + 1$$

### Hàm kích hoạt:

Hàm kích hoạt là một hàm số nhận vào một giá trị đầu vào và kết quả là một giá trị có miền giá trị nằm trên một khoảng (hay nửa khoảng) nào đó. Một số các hàm kích hoạt phổ biến có thể kể đến

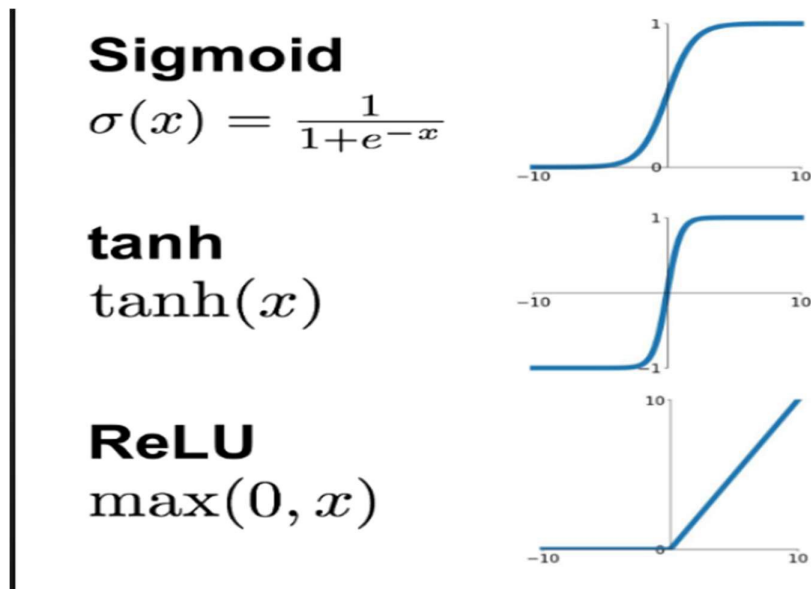
đó là Sigmoid, Tanh, ReLU. Hàm kích hoạt rất quan trọng bởi vì nó sẽ tăng khả năng dự đoán của mạng nơ-ron và giúp mô hình

học được các quan hệ phi tuyến phức tạp tiềm ẩn trong dữ liệu. Thông thường hàm kích hoạt sử dụng ở giữa các tầng convolution và pooling là hàm Relu. [4]



Hình 2.4 Ví dụ về việc sử dụng hàm kích hoạt ReLU

Hàm ReLU thường được ưa chuộng vì tính toán đơn giản, giúp hạn chế tình trạng gradient biến mất (vanishing gradient) mà hàm Sigmoid hay tanh thường gặp, và cũng cho kết quả tốt hơn. Đạo hàm của ReLU là 1 cho các giá trị lớn hơn 0. Bởi vì nhân 1 với chính nó nhiều lần vẫn cho 1, điều này về cơ bản giải quyết vấn đề gradient biến mất. Không thể phân biệt giá trị âm của hàm ReLU vì nó bằng 0. Do đó, các đạo hàm của giá trị âm chỉ đơn giản được đặt thành 0. Relu cũng như những hàm kích hoạt khác, được đặt ngay sau tầng convolution.



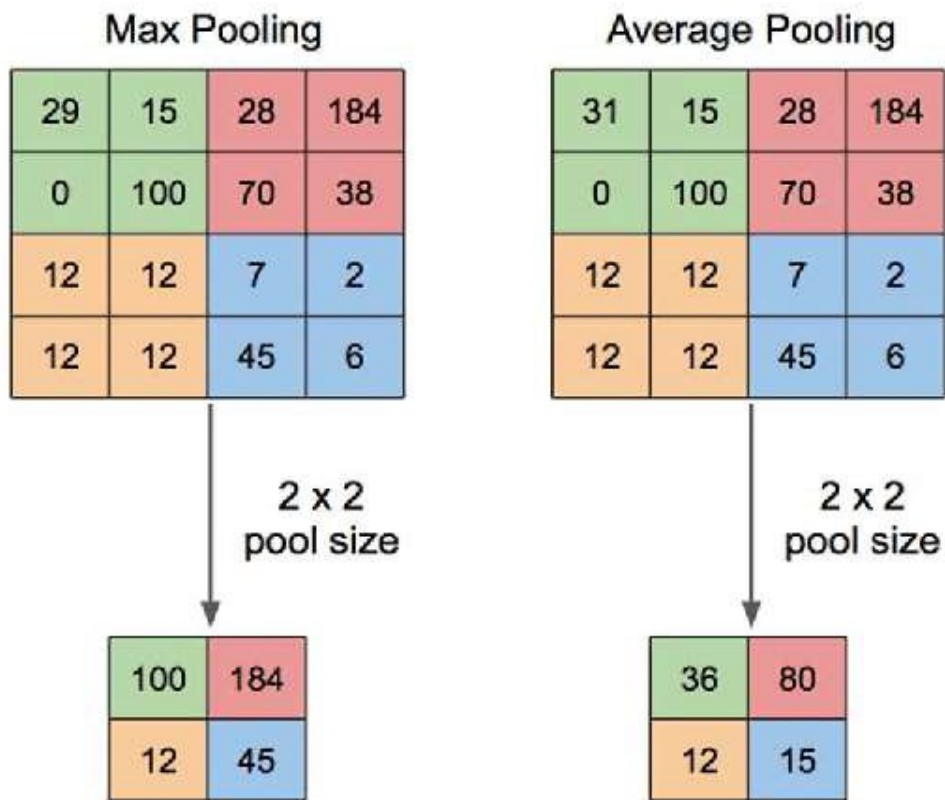
Hình 2.5 Đồ thị và công thức của một số hàm kích hoạt

#### Pooling layer:

Lớp Pooling được sử dụng ngay sau lớp convolutional và hàm kích hoạt phi tuyến. Pooling giúp cho mạng giảm số lượng tham số, từ đó giúp đơn giản hóa quá trình tính toán của CNN và qua đó góp phần giải quyết vấn đề overfitting khi huấn luyện mạng và giảm thời gian huấn luyện.

Có nhiều loại pooling nhưng được sử dụng phổ biến nhất là Max-pooling và Average-pooling. Max-pooling hoạt động bằng cách thực hiện lấy giá trị lớn nhất trong kích thước Filter mà chúng ta xét. Điều này giúp giữ lại các đặc trưng mạnh mẽ nhất trong mỗi vùng và loại bỏ các giá trị nhỏ hơn. Max-pooling giúp làm cho mạng trở nên ít nhạy cảm hơn với các biến dạng và dịch chuyển nhỏ trong ảnh đầu vào, bởi vì các đặc trưng mạnh mẽ vẫn được giữ lại dù có sự thay đổi nhỏ. Nhược điểm của Max-pooling là có thể làm mất đi một số thông tin quan trọng. Còn Average-pooling hoạt động bằng cách lấy trung bình cộng của tất cả các giá trị của Filter. Giúp giữ lại nhiều thông tin hơn từ đặc trưng gốc. Có thể giúp mô hình học được các mối quan hệ phức tạp giữa các đặc điểm. Nhược điểm của Average-pooling là có thể khiến mô hình trở nên tốn kém về mặt tính toán hơn so với Max-pooling.





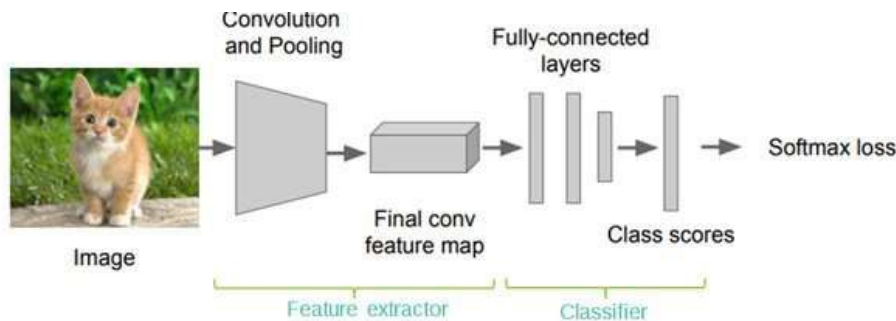
Hình 2.4 Ví dụ về Max-pooling và Average-pooling

#### Lớp Normalization:

Lớp Normalization (Norm) là lớp giúp chuẩn hóa dữ liệu đầu ra cho các lớp trong CNN trước khi được truyền đi tiếp. Trong những kiến trúc CNN lớn và phức tạp, lớp Norm sẽ chuẩn hóa các giá trị neural trước khi chúng được truyền đến hàm Relu. Hàm Relu tuy giúp rút ngắn thời gian huấn luyện, nhưng nếu không điều chỉnh trọng số phù hợp, hàm Relu sẽ rất dễ gặp phải vấn đề "dying Relu" khiến cho mạng trở nên chậm hơn khi huấn luyện. Lớp Norm lúc này sẽ chuẩn hóa và tạo ra các giá trị tích chập phù hợp để tránh cho Relu rơi vào giá trị 0. Tránh việc gradient xấp xỉ bằng 0 khiến cho tốc độ học của mạng trở nên rất chậm. Các Norm thường dùng bao gồm Batch Normalization và Layer Normalization. Batch Normalization là lựa chọn phổ biến cho các mô hình CNN với batch size lớn. Layer Normalization thích hợp cho các mô hình tuần tự và khi batch size nhỏ.

#### Fully connected Layer:

Tầng cuối cùng của mô hình CNN trong bài toán phân loại ảnh là tầng fully connected layer. Sau khi ảnh được truyền qua nhiều convolutional layer và pooling layer thì model đã học được tương đối các đặc điểm của ảnh thì tensor của output của layer cuối cùng sẽ được là phẳng thành vector và đưa vào một lớp được kết nối như một mạng nơ-ron. Với FC layer được kết hợp với các tính năng lại với nhau để tạo ra một mô hình. Cuối cùng sử dụng softmax hoặc sigmoid để phân loại đầu ra. Đầu ra là một vec-tơ chứa xác suất của các đối tượng cần dự đoán. Ví dụ trong một bài toán phân loại ảnh với có 10 lớp, tầng fully connected layer sẽ chuyển ma trận đặc trưng của tầng trước thành vector có 10 chiều thể hiện xác suất của 10 lớp tương ứng.



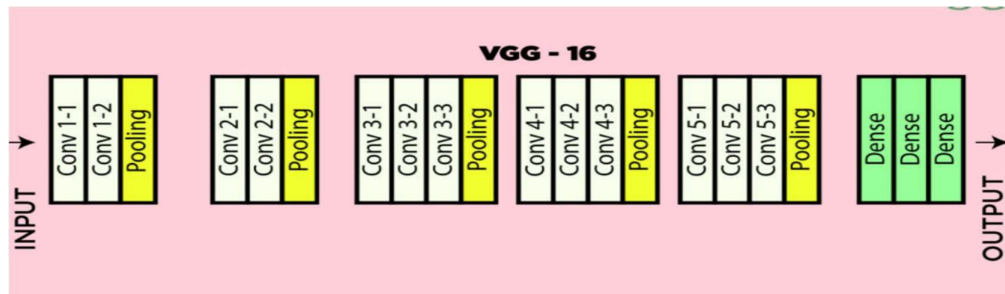
Hình 2.5 Minh họa Fully Connected Layer

### Mô hình VGG16:

Mô hình VGG16 là một kiến trúc của mạng CNN nổi tiếng, được phát triển bởi nhóm nghiên cứu tại Visual Geometry Group (VGG) thuộc Đại học Oxford [5]. VGG16 thường được sử dụng cho các nhiệm vụ phân loại hình ảnh. Mạng bao gồm 16 lớp nơ-ron, mỗi lớp hoạt động để xử lý thông tin hình ảnh tăng dần và cải thiện độ chính xác của dự đoán. Mặc dù đơn giản so với các kiến trúc gần đây hơn, VGG16 vẫn là lựa chọn phổ biến cho nhiều ứng dụng deep learning do tính linh hoạt và hiệu suất tuyệt vời của nó. Mạng VGG16 được đào tạo trên hơn một triệu hình ảnh từ cơ sở dữ liệu trực quan ImageNet và có thể phân loại hình ảnh thành 1.000 lớp khác nhau với độ chính xác trong thử nghiệm top 5 là 92,7%. VGG16 có thể được áp dụng để xác định xem hình ảnh có chứa một số vật phẩm, động vật, thực vật và hơn thế nữa.

VGG16 có tổng cộng 16 lớp chính (do đó có tên là VGG16, bao gồm 13 lớp tích chập (convolutional layers) và 3 lớp kết nối đầy đủ (fully connected layers)). Các khối này được tổ chức thành các khối, mỗi khối bao gồm các lớp tích chập (convolutional layer) và 1 lớp pooling. Mỗi khối tích chập sử dụng các bộ filter kích thước 3x3 với số lượng

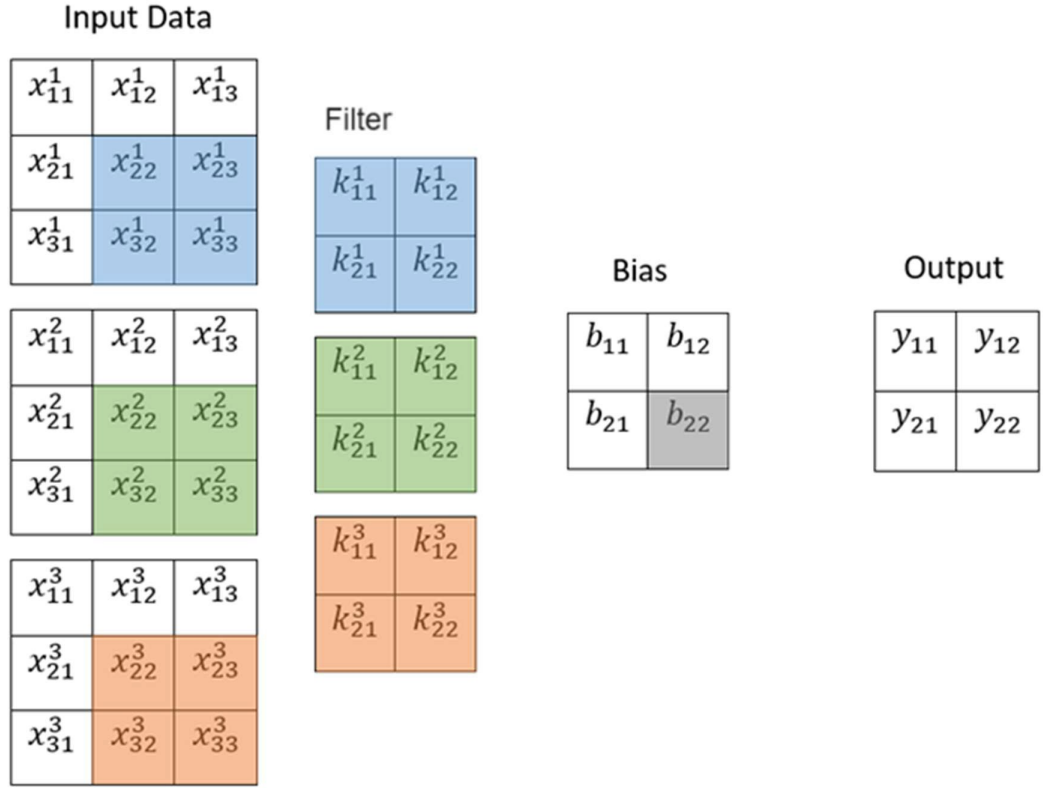
kênh đầu ra tăng gấp đôi sau mỗi khối. Bắt đầu với 64 bộ filter cho khối đầu tiên và tăng lên 128, 256, 512, sau đó là 512 cho các khối tiếp theo. Sử dụng hàm kích hoạt ReLU sau mỗi lớp tích chập. Áp dụng lớp max pooling 2x2 sau mỗi khối để giảm kích thước không gian của dữ liệu và giảm bớt thông số mô hình. Sau các khối tích chập, VGG16 sử dụng 3 lớp fully connected với kích thước lần lượt là 4096, 4096 và 1000. Mỗi lớp kết nối đầy đủ được theo sau bởi một hàm kích hoạt ReLU và một lớp dropout với tỷ lệ 0.5 để tránh quá phù hợp. Hàm kích hoạt softmax được áp dụng cho đầu ra của lớp fully connected thứ ba để phân loại. Lớp đầu ra có 1000 nút, tương ứng với 1000 lớp trong tập dữ liệu ImageNet.



Hình 2.6 Kiến trúc mô hình VGG16

#### Cách thức hoạt động của mạng CNN:

Ví dụ với đầu vào là một ảnh có kích thước 3x3 với 3 kênh màu và bộ filter có kích thước 2x2 với kênh màu tương ứng như hình sau:



Hình 2.7 Ví dụ cách thức hoạt động của CNN

Với đầu vào là một bức ảnh có kích thước là  $3 \times 3$  với 3 kênh màu được biểu diễn dưới dạng ma trận có kích thước tương ứng, với mỗi phần tử trong ma trận tương ứng với giá trị cường độ pixel của ảnh tại vị trí đó và bộ filter chứa trọng số cũng được biểu diễn dưới dạng ma trận  $2 \times 2$  có cùng kênh màu. Với  $\text{Stride} = 1$  và  $\text{Padding} = 0$ , đầu tiên bộ filter sẽ trượt trên bức ảnh và thực hiện tích chập bằng cách di chuyển qua từng vị trí trên ảnh đầu vào và thực hiện phép nhân điểm tương ứng giữa các phần tử của bộ lọc và vùng ảnh tương ứng, sau đó cộng lại để tạo ra một giá trị mới trong bản đồ đặc trưng, sau đó tiếp tục di chuyển bộ filter đến vị trí tiếp theo dựa trên  $\text{Stride}$ , thực hiện tiếp tục phép tích chập đi qua toàn bộ ảnh, sau khi thực hiện phép tích chập, ta thêm vào một giá trị bias. Sau đó cho đi qua một hàm kích hoạt để thu được một future map chứa các kết quả đã tính toán. Ta có một công thức tính toán cho output của phép tích chập như sau:

$$y = \sum (x_{ij} \times f_{ij}) + b$$

Hình 2.8 Công thức tính toán output của Convolution

Trong đó:

- y là output của phép tích chập.
- x là input.
- f là filter.
- b là bias

Ví dụ theo Hình 11. ta có thể tính được giá trị của y11 như sau:

$$\begin{aligned}
 y_{11} &= \begin{matrix} 1 & 1 \\ 1 & 1 \end{matrix} * \begin{matrix} 1 & 1 \\ 1 & 1 \end{matrix} + \begin{matrix} 1 & 1 \\ 1 & 1 \end{matrix} * \begin{matrix} 1 & 1 \\ 1 & 1 \end{matrix} + \begin{matrix} 1 & 1 \\ 1 & 1 \end{matrix} * \begin{matrix} 1 & 1 \\ 1 & 1 \end{matrix} + \begin{matrix} 1 & 1 \\ 1 & 1 \end{matrix} * \begin{matrix} 1 & 1 \\ 1 & 1 \end{matrix} \\
 &= \begin{matrix} 12 & 12 & 21 & 21 \\ 22 & 22 & & \end{matrix} \\
 &+ \begin{matrix} 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \end{matrix} + \begin{matrix} 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \end{matrix} + \begin{matrix} 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \end{matrix} + \begin{matrix} 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 \end{matrix} \\
 &= \begin{matrix} 12 & 12 & 21 & 21 \\ 22 & 22 & & \end{matrix} \\
 &+ \begin{matrix} 3 & 3 & 3 & 3 \\ 3 & 3 & 3 & 3 \end{matrix} + \begin{matrix} 3 & 3 & 3 & 3 \\ 3 & 3 & 3 & 3 \end{matrix} + \begin{matrix} 3 & 3 & 3 & 3 \\ 3 & 3 & 3 & 3 \end{matrix} + \begin{matrix} 3 & 3 & 3 & 3 \\ 3 & 3 & 3 & 3 \end{matrix} \\
 &= \begin{matrix} 11 & 11 & 12 & 12 & 21 & 21 & 22 & 22 \\ & & & & & & & \end{matrix} \\
 &+ \begin{matrix} 11 & 11 & & & & & & \end{matrix}
 \end{aligned}$$

Và các y12, y21, y22 được tính toán theo cách tương tự.

Sau khi thực hiện tính toán ở tầng tích chập, dữ liệu sẽ được chuyển tiếp đến tầng pooling để thực hiện giảm kích thước của các future map. Thông thường sử dụng Max-pooling hoặc Average-pooling để giảm bớt thông số và tăng tính tương tác của mạng. Với Max-pooling thì lấy giá trị lớn nhất trong vùng đang xét của bộ filter, còn Average-pooling thì lấy giá trị trung bình. Lớp pool sử dụng nhiều bộ filter khác nhau để xác định các phần khác nhau của hình ảnh như cạnh, góc, thân, lông, mắt và mỏ của ảnh. Quá trình Convolution + ReLU – Pooling có thể lặp đi lặp lại nhiều lần để tạo ra các future map ở cấp độ cao hơn. Mỗi lần lặp, kích thước của future map sẽ giảm dần, nhưng số lượng future map có thể tăng lên. Sau đó các future map sẽ được chuyển đổi từ một ma trận nhiều chiều thành một ma trận một chiều gọi là Flattening và đưa đến tầng Fully Connected. Tiếp theo từ dữ liệu làm phẳng, mà mỗi đầu vào đó được kết nối đến tất cả nơ-ron để tính toán các giá trị đầu ra dựa trên các trọng số và bias đã học. Thông thường, khi tất cả các tính năng được kết nối với lớp

FC, nó có thể gây ra hiện tượng overfitting trong tập dữ liệu huấn luyện. Overfitting xảy ra khi một mô hình cụ thể hoạt động quá tốt trên dữ liệu huấn luyện gây ra tác động tiêu cực đến hiệu suất của mô hình khi được sử dụng trên dữ liệu mới. Để giải quyết vấn đề này một lớp “Dropout” được thêm vào, trong đó một số nơ-ron bị loại khỏi mạng trong quá trình huấn luyện để giảm kích thước mô hình. Sau khi thông qua các lớp FC, đầu ra của lớp cuối cùng có nơ-ron sẽ bằng với số tổng lớp cần phân đối với bài toán phân loại hình ảnh, và đi qua một hàm kích hoạt Softmax hoặc Sigmoid để phân bố xác suất cuối cùng. Lớp có xác suất cao nhất là lớp cuối cùng của ảnh đầu vào.

#### **Cách chọn tham số cho CNN: [6]**

- Số các convolution layer: càng nhiều các convolution layer thì hiệu suất càng được cải thiện. Sau khoảng 3 hoặc 4 layer, các tác động được giảm một cách đáng kể
- Filter size: thường filter theo size  $5 \times 5$  hoặc  $3 \times 3$ .
- Pooling size: thường là  $2 \times 2$  hoặc  $4 \times 4$  cho ảnh đầu vào lớn.

#### **2.1.2. Các phép tích chập tối giản (Depthwise Convolution và Point-wise Convolution)**

*[Nội dung phần này đang được cập nhật/viết tiếp...]*

#### **2.1.3. Kiến trúc khối Full-Residual Point-Depth-Point (FR-PDP)**

*[Nội dung phần này đang được cập nhật/viết tiếp...]*

#### **2.1.4. Cấu trúc xương sống cơ giãn kênh hình dấu tích (Tick-shape Backbone) và mô hình TickNet-small**

*[Nội dung phần này đang được cập nhật/viết tiếp...]*

#### **2.1.5. Cơ chế lan truyền đặc trưng và cách thức hoạt động của Tick-Nets**

*[Nội dung phần này đang được cập nhật/viết tiếp...]*

### **2.2. CƠ CHẾ CHÚ Ý (ATTENTION MECHANISM):**

Cơ chế chú ý trong Neural Network có xu hướng bắt chước sự chú ý về mặt nhận thức của con người. Một đặc tính quan trọng của thị giác con người là người ta không cố gắng xử lý toàn bộ cảnh vật trong cùng một lúc. Thay vào đó, con người khai thác chuỗi những cái nhìn thoáng qua một phần và tập trung có chọn lọc vào những

điểm nổi bật các bộ phận để nắm bắt cấu trúc hình ảnh tốt hơn. Đó cũng là ý tưởng chính cho cơ chế này nhằm nhấn mạnh những phần quan trọng của thông tin và cố gắng giảm bớt những phần không liên quan. Vì bộ nhớ làm việc ở cả con người và máy móc đều có hạn nên quá trình này là chìa khóa để không làm quá tải bộ nhớ của hệ thống. Trong học sâu, sự chú ý có thể được hiểu là một vectơ có trọng số quan trọng.

Trong thị giác máy tính (CV), cơ chế chú ý cho phép mô hình tập trung vào các vùng cụ thể của hình ảnh và loại bỏ đi những vùng không quan trọng. Ở mức đơn giản, cơ chế Attention trong phân loại hình ảnh hoạt động bằng cách tạo ra một trọng

số cho từng phần của hình ảnh đầu vào. Các trọng số này cho biết mức độ quan trọng của mỗi phần vùng hình ảnh đối với quá trình phân loại. Trong quá trình huấn luyện, mô hình học từ dữ liệu để xác định các vùng quan trọng này. [7]

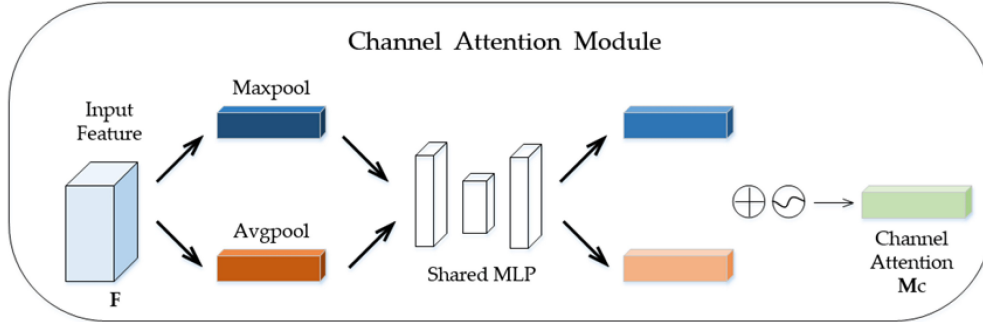
#### **2.2.1. Cơ chế chú ý theo kênh Squeeze-and-Excitation (SE) mặc định trong TickNets**

*[Nội dung phần này đang được cập nhật/viết tiếp...]*

#### **2.2.2. Mô-đun chú ý theo kênh (Channel Attention Module - CAM) của CBAM**

Channel Attention Module (CAM) là một thành phần trong các mô hình mạng nơ-ron sâu, đặc biệt là trong lĩnh vực xử lý ảnh và thị giác máy tính, nhằm cải thiện khả năng chú ý của mạng nơ-ron tới các kênh thông tin quan trọng trong một tập hợp các đặc trưng. CAM được thiết kế để gán trọng số khác nhau cho các kênh đặc trưng (feature channel), giúp mạng nơ-ron có thể tập trung vào các kênh quan trọng hơn trong quá trình xử lý dữ liệu. Điều này cải thiện khả năng của mạng nơ-ron trong việc phát hiện và phân loại các đặc trưng có ý nghĩa hơn từ dữ liệu đầu vào.

CAM hoạt động bằng cách tính toán một bản đồ trọng số chú ý (attention map) cho từng kênh đặc trưng. Mỗi kênh của feature map được coi như là một feature detector, channel attention tập trung vào “điều gì” có ý nghĩa đối với ảnh đầu vào. Quá trình hoạt động của CAM thường bao gồm các bước sau theo hình sau:



Hình 2.9 Mô hình hoạt động của CAM

CAM nhận đầu vào là một input feature có kích thước  $H \times W \times C$ , với  $H$  là chiều cao,  $W$  là chiều rộng và  $C$  là số kênh. Sau đó thực hiện thu gom thông tin toàn cục bằng cách sử dụng kết hợp cả hoạt động Global Max-pooling và Global

Average-pooling để tạo ra hai bộ vector đặc trưng khác nhau là:  $F_{avg}$  và  $F_{max}$ . Sau đó hai bộ vector đại diện được đưa qua một mạng con nhỏ multi-layer perceptron (MLP). Thường thì mạng con này gồm hai lớp fully connected với một hàm kích hoạt phi

tuyến ReLU ở giữa. Để giảm số lượng tham số cần học, kích thước của lớp kích hoạt ẩn được đặt bằng  $\frac{C}{r} \times 1 \times 1$ , với  $r$  là tỉ lệ giảm. Kết quả của MLP được kết hợp bằng cách sử dụng phép tính tổng theo phần tử và chuyển qua hàm kích hoạt sigmoid để tạo ra một vector trọng số gọi là channel attention map  $MC(F) \in \mathbb{R}^{C \times 1 \times 1}$ . Công thức tính toán cho quá trình đó được biểu diễn như sau:

$$MC(F) = (\text{MLP}(\text{Avgpool}(F)) + \text{MLP}(\text{Maxpool}(F))) \\ = (W_1(W_0(F_{avg})) + W_1(W_0(F_{max})))$$

Trong đó :

$\sigma$  : là hàm kích hoạt sigmoid.

$W_1, W_0$ : lần lượt là trọng số của MLP.

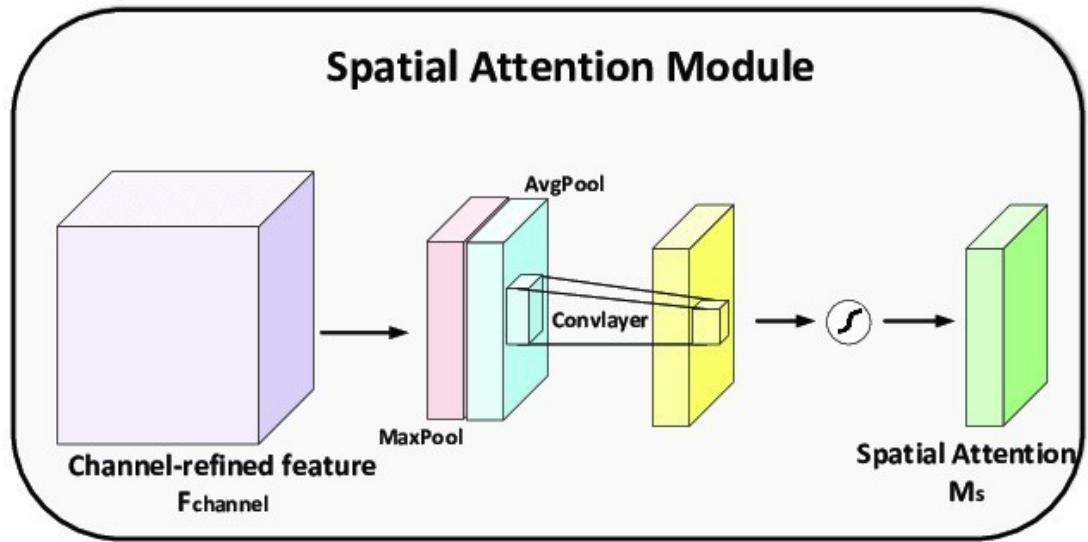
### 2.2.3. Mô-đun chú ý theo không gian (Spatial Attention Module - SAM) của CBAM

Spatial Attention Module (SAM) được sử dụng để cải thiện hiệu suất của mô hình bằng cách tập trung vào các vị trí quan trọng trong feature map. Khác với Channel Attention Module (CAM) tập trung vào các kênh, SAM tập trung vào không gian của các feature maps, xác định các vùng quan trọng trong ảnh dựa vào quan hệ của chúng, nó cho phép mô hình học cách chú ý đến các vùng không



gian cụ thể trong dữ liệu có liên quan nhất đến nhiệm vụ đang thực hiện.

SAM hoạt động bằng cách tạo ra một bản đồ chú ý không gian (spatial attention map), giúp xác định và nhấn mạnh các vùng quan trọng trong ảnh đầu vào, dựa trên các đặc điểm không gian. Điều này giúp mô hình tập trung nhiều hơn vào các vùng quan trọng và bỏ qua những vùng không quan trọng, từ đó cải thiện khả năng nhận dạng và phân loại. Quá trình SAM thực hiện theo hình sau:



Hình 2.10 Mô hình hoạt động của SAM

SAM thường bắt đầu bằng việc tổng hợp thông tin từ tất cả các kênh trong đầu vào để tạo ra một bản đồ không gian duy nhất. Để tạo ra một bản đồ không gian duy nhất, thông thường, hai phương pháp phổ biến là Global Average-pooling (GAP) và Global Max-pooling (GMP) được sử dụng. Các bản đồ kết quả từ GAP và GMP được kết hợp lại để tạo ra một bản đồ chú ý không gian duy nhất. Thông thường, các giá trị này được kết hợp bằng cách nối chúng hoặc thực hiện phép cộng element-wise. Sau đó, bản đồ kết hợp được đưa qua một lớp convolution với bộ filter có kích thước  $k \times k$  (thông thường là  $7 \times 7$ ), sau đó đi qua một hàm kích hoạt sigmoid, kết quả là một bản đồ chú ý không gian 2D (spatial attention map)  $MS(F)$ . Công thức tính toán bản đồ chú ý không gian là:

$$MS(F) = (f^{7 \times 7} ([AvgPool(F); MaxPool(F)]))$$

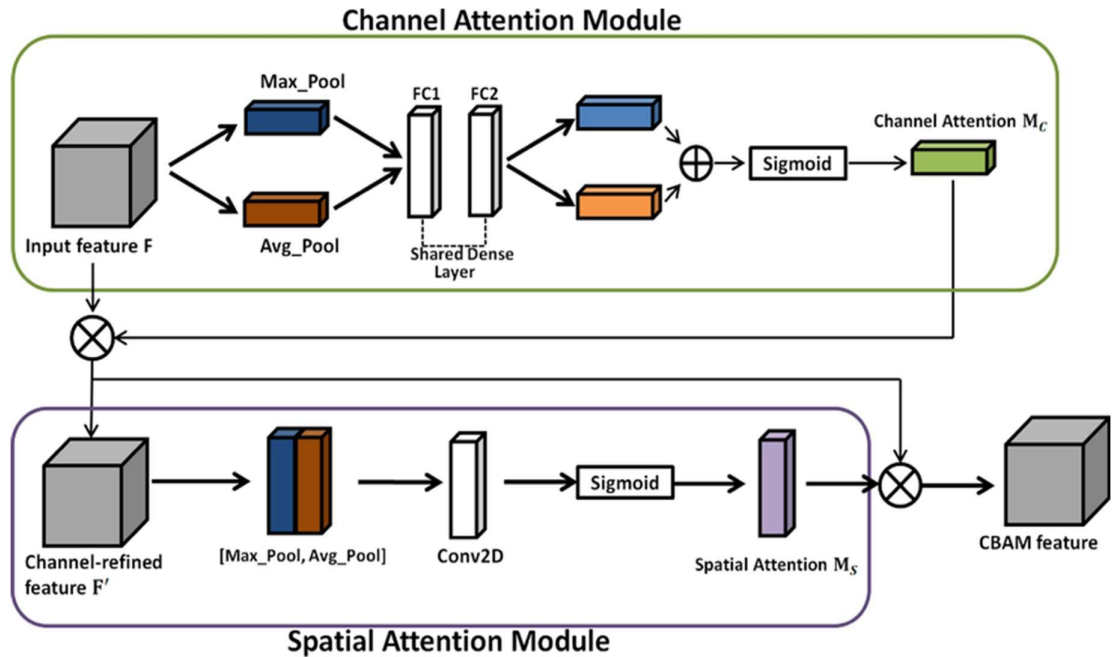
Trong đó:

$f^{7 \times 7}$  : Biểu thị phép toán tích chập với kích thước bộ lọc  $7 \times 7$ .

$f$  : Biểu thị hàm kích hoạt sigmoid.

#### 2.2.4. Mô-đun chú ý khối tích chập CBAM hoạt động tuần tự (CAM + SAM)

Convolutional Block Attention Module (CBAM) là một kỹ thuật tiên tiến trong lĩnh vực học sâu, được giới thiệu nhằm cải thiện hiệu suất của các mạng nơ-ron tích chập (Convolutional Neural Networks - CNN). CBAM tích hợp các cơ chế chú ý (attention mechanisms) vào các khối tích chập để giúp mạng nơ-ron tập trung vào các đặc trưng quan trọng của dữ liệu đầu vào. CBAM là sự kết hợp giữa hai mô-đun là mô-đun chú ý kênh (Channel Attention Module – CAM) và mô-đun chú ý không gian (Spatial Attention Module – SAM). Những mô-đun này hoạt động tuần tự để tăng cường khả năng trích xuất đặc trưng của mạng nơ-ron.



Hình 2.11 Mô hình CBAM

Đầu tiên với input feature  $F$  đầu vào sẽ qua CAM thực hiện các phép tính liên quan cho ra bản đồ chú ý không gian (channel attention map)  $M_c(F)$ . Sau đó  $M_c(F)$  được nhân theo từng phần tử với input feature ban đầu  $F$  thu được future map  $F'$ . Sau đó future map  $F'$  có được sau khi đi qua CAM sẽ được đưa vào đầu vào của SAM thực hiện, thu được kết quả là một bản đồ chú ý không gian 2D (spatial attention map)  $M_s(F')$ . Bản đồ chú ý không gian  $M_s(F')$  được nhân theo từng phần tử đầu vào  $F$  để

cho ra đầu ra cuối cùng  $F''$  hay CBAM feature. Quá trình thực hiện

tính toán được biểu diễn dưới dạng công thức như sau:

$$F' = Mc(F) \quad F' \cdot F' = Ms(F') \quad F'$$

Trong đó:

$\cdot$  : là phép nhân theo từng phần tử (element-wise multiplication).

Một trong những lợi ích chính của CBAM đặc biệt là khả năng áp dụng bản đồ chú ý dọc theo cả kênh và chiều không gian. Điều này cho phép mô hình tập trung hiệu quả hơn vào cả các tính năng cấp cao (chẳng hạn như đối tượng hoặc hình dạng) và các tính năng cấp thấp (chẳng hạn như các cạnh hoặc kết cấu) cùng một lúc. Điều này có thể đặc biệt hữu ích trong các tác vụ như phát hiện đối tượng, trong đó cả hai loại tính năng đều quan trọng để xác định chính xác đối tượng. [9][10][11]

### 2.3. PHƯƠNG PHÁP NGHIÊN CỨU

Đề tài được triển khai dựa trên sự kết hợp giữa phương pháp nghiên cứu lý thuyết và phương pháp nghiên cứu thực nghiệm.

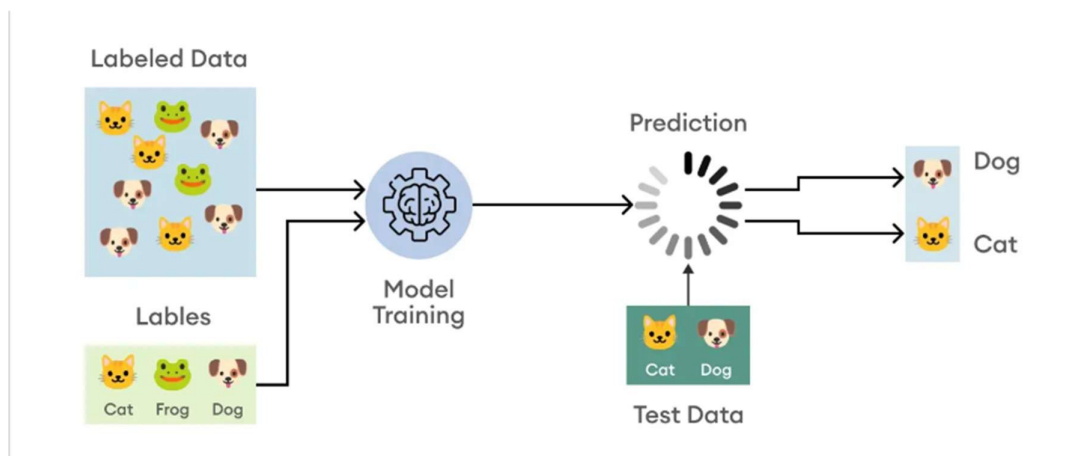
**Về phương pháp lý thuyết,** nghiên cứu tiến hành khảo cứu hệ thống các công trình khoa học liên quan bao gồm bài báo gốc về kiến trúc TickNets, cơ chế chú ý CBAM (Woo et al., 2018) và cơ chế Squeeze-and-Excitation (Hu et al., 2018). Trên cơ sở đó, đề tài phân tích cấu trúc luồng đặc trưng trong khối FR-PDP, vai trò của các phép tích chập tối giản Depthwise và Pointwise, cũng như cơ chế hoạt động tuần tự của CBAM. Từ kết quả phân tích lý thuyết, nghiên cứu xây dựng các giả thuyết khoa học về mối quan hệ giữa vị trí chèn cơ chế chú ý và hiệu năng mô hình, đặc biệt là hiện tượng chồng chéo đặc trưng không gian khi đặt Spatial Attention Module cùng mức cục bộ với Depthwise Convolution bên trong khối FR-PDP.

**Về phương pháp thực nghiệm,** nghiên cứu áp dụng phương pháp thiết kế thí nghiệm đối chứng với 3 mô hình (SE Baseline, CBAM-Local và CBAM-Hook) được huấn luyện trên cùng điều kiện để kiểm chứng các giả thuyết đã đặt ra. Bốn bộ dữ liệu được lựa chọn đại diện cho hai nhóm bài toán: nhóm chuẩn học thuật (Fashion-MNIST, CIFAR-10) và nhóm ứng dụng thực tiễn (PlantVillage, Chest-Xray-Pneumonia). Hiệu năng của các mô hình được đánh giá thông qua bộ độ đo chuẩn mực gồm Accuracy, Precision, Recall, F1-Score và ma trận nhầm lẫn (Confusion Matrix), kết hợp với trực quan hóa bản đồ nhiệt Grad-CAM để giải thích vùng chú ý của mô hình. Toàn bộ quá trình thực nghiệm được triển khai trên nền tảng Kaggle Notebooks sử dụng ngôn ngữ Python với framework TensorFlow/Keras. Chi tiết về môi trường, dữ liệu, cấu hình huấn luyện và kết quả thực nghiệm được trình bày đầy đủ tại Chương 3.

## CHƯƠNG 3: BÀI TOÁN PHÂN LOẠI HÌNH ẢNH VÀ ĐỀ XUẤT CẢI TIẾN PHÂN CẤP

### 3.1. GIỚI THIỆU BÀI TOÁN PHÂN LOẠI HÌNH ẢNH

Phân loại hình ảnh là một trong những bài toán cơ bản và quan trọng nhất trong lĩnh vực thị giác máy tính (CV). Mục tiêu chính của bài toán đó chính là phân loại một hình ảnh đầu vào (input) thành một nhãn (label) cho đầu ra (output). Dữ liệu đầu vào của bài toán là một bức ảnh. Một ảnh được biểu diễn bằng ma trận các giá trị. Mô hình phân lớp sẽ phải dự đoán được lớp của ảnh từ ma trận điểm ảnh này. Hay có thể nói một cách đơn giản, bài toán này đặt ra câu hỏi: *"Một hình ảnh thuộc vào nhóm nào trong số các nhóm đã định trước?"*. Ví dụ, với một tập các hình ảnh, mỗi hình ảnh mô tả một con mèo hoặc một con chó, thuật toán sẽ *"quan sát"* toàn bộ dữ liệu và dựa trên hình dạng, màu sắc để hình thành giả thuyết liên quan đến nội dung của ảnh. Kết quả thu được từ tập dữ liệu ban đầu, các hình ảnh chó/ mèo sẽ được phân loại một cách tự động.



Hình 3.1 Ví dụ về bài toán phân loại ảnh

### 3.2. ĐỀ XUẤT MÔ HÌNH TICKNETS KẾT HỢP CƠ CHẾ CHÚ Ý PHÂN CẤP (HIERARCHICAL ATTENTION):

#### 3.2.1. Thiết kế kiến trúc đề xuất TickNet-small CBAM-Hook

Theo các nghiên cứu thì các phương pháp học sâu (DL) đã được chứng minh là có thể mang lại hiệu quả cao cho việc giải quyết về

bài toán phân loại hình ảnh. Trong đó mô hình CNN là mô hình phổ biến nhất và có ảnh hưởng nhất.

Mặc dù CNN đã đạt được nhiều thành công trong các bài toán thị giác máy tính, nhưng nó vẫn tồn tại một số hạn chế như thiếu khả năng tập trung vào các vùng quan trọng vì các lớp convolution xử lý toàn bộ ảnh một cách đồng đều dẫn đến việc mô hình có thể bỏ qua những vùng quan trọng trong ảnh. Vì vậy, cơ chế chú ý (Attention Mechanism) được nghiên cứu và đưa vào mô hình nhằm khắc phục những vấn đề mà mạng CNN gặp phải. Có nhiều cơ chế chú ý có thể được áp dụng vào mô hình CNN, nhưng trong bài nghiên cứu này sẽ sử dụng cơ chế chú ý CBAM. CBAM (Convolutional Block Attention Module) thường được thêm vào sau các lớp convolution trong mô hình CNN để tăng cường khả năng chú ý của mạng đối với các đặc trưng quan trọng. Cụ thể, CBAM thường được chèn vào sau mỗi lớp convolutional block hoặc sau các lớp cụ thể nơi cần cải thiện khả năng chú ý. Các vị trí phổ biến để thêm CBAM:

- **Sau Các Lớp Convolution:** Thêm CBAM ngay sau mỗi lớp convolution để mạng học cách tập trung vào các kênh và vùng không gian quan trọng ngay sau khi các đặc trưng được trích xuất.
- **Sau Mỗi Convolutional Block:** Thêm CBAM sau mỗi block của các lớp convolution (thường là một nhóm các lớp convolution, BatchNorm, và ReLU) để tổng hợp các đặc trưng ở mức cao hơn và cải thiện chú ý.
- **Trước Các Lớp Kết Nối Đầy Đủ (Fully Connected Layers):** Thêm CBAM trước khi dữ liệu được đưa vào các lớp fully connected để tối ưu hóa các đặc trưng quan trọng trước khi thực hiện phân loại cuối cùng.

Hình 3.2 Mô hình CNN kết hợp CBAM

Input  
Conv  
CBAM  
Pooling  
Flatten  
FC  
Output

### 3.2.2. Cơ chế chú ý phân cấp (Giữ SE nội khối FR-PDP, chèn CBAM ngoại khối tại các điểm nối Hooking points và cuối mạng)

*[Nội dung phần này đang được cập nhật/viết tiếp...]*

### 3.2.3. Phân tích toán học giải quyết sự chồng chéo đặc trưng không gian giữa Depthwise Convolution và Spatial Attention Module (SAM)

*[Nội dung phần này đang được cập nhật/viết tiếp...]*

## 3.3. HIỆN THỰC GIẢI PHÁP VÀ THỰC NGHIỆM:

### 3.3.1. Môi trường thực nghiệm và xây dựng mô hình:

Quá trình xây dựng mô hình và huấn luyện bao gồm các giai đoạn:



*Hình 3.3 Quá trình xây dựng và huấn luyện mô hình*

#### 3.3.1.1. Môi trường thực nghiệm (Nền tảng Kaggle, GPU NVIDIA T4/P100, TensorFlow và Keras)

Ngôn ngữ lập trình để thực hiện quá trình thực nghiệm là Python với các thư viện hỗ trợ như Keras, TensorFlow, Pytorch, Numpy, Scikit-learn...

Quá trình xây dựng và huấn luyện mô hình được thực hiện trên nền tảng điện toán đám mây Kaggle Notebooks chạy hệ điều hành Linux, với dung lượng ổ đĩa khả dụng khoảng 73.0 GB, 13GB–30GB RAM hệ thống và bộ tăng tốc card đồ họa GPU NVIDIA T4 (15GB VRAM) hoặc GPU P100 (16GB VRAM).

#### 3.3.1.2. Mô tả 4 bộ dữ liệu thực nghiệm:

##### 3.3.1.2.1. Nhóm dữ liệu chuẩn học thuật: Fashion-MNIST và CIFAR-10

Chuẩn bị tập dữ liệu cho thực nghiệm bao gồm tập huấn luyện và tập kiểm tra. Các tập dữ liệu được chọn để thực hành nghiên cứu bao gồm Fashion-MNIST và CIFAR-10.

- **Fashion-MNIST:** là tập dữ liệu chứa các hình ảnh thuộc 10 lớp, mỗi lớp có 6,000 ảnh trong tập huấn luyện và 1,000 ảnh trong tập kiểm tra. Do đó, tập huấn luyện và tập kiểm tra sẽ chứa tổng cộng lần lượt 60,000 và 10,000 ảnh.

Các ảnh trong Fashion-MNIST tương ứng với các lớp: Áo phông/T-shirt (T-shirt/top), Quần (Trouser), Áo len (Pullover), Váy (Dress), Áo khoác (Coat), Dép (Sandal), Áo sơ mi (Shirt), Giày thể thao

(Sneaker), Túi xách (Bag), Bốt/giày cổ cao (Ankle boot). Mỗi hình ảnh có kích thước 28x28 pixel và là ảnh xám (grayscale).

**Bảng 3.1** Bảng mô tả dữ liệu tập dữ liệu MNIST-Fashion

| Thuộc tính              | Mô tả                                                                                                               |
|-------------------------|---------------------------------------------------------------------------------------------------------------------|
| Số lượng ảnh            | 70,000                                                                                                              |
| Số lớp                  | 10                                                                                                                  |
| Kích thước ảnh          | 28x28 pixels                                                                                                        |
| Định dạng               | Grayscale (ảnh đen trắng)                                                                                           |
| Số lượng tập huấn luyện | 60,000                                                                                                              |
| Số lượng tập kiểm tra   | 10,000                                                                                                              |
| Tên các lớp             | 0: T-shirt/top, 1: Trouser, 2: Pullover, 3: Dress, 4: Coat, 5: Sandal, 6: Shirt, 7: Sneaker, 8: Bag, 9: Ankle boot. |

- **CIFAR-10:** là tập dữ liệu bao gồm 60,000 hình ảnh có kích thước 32x32 pixel và là ảnh màu (RGB) được chia thành 10 lớp, với 6,000 hình ảnh mỗi lớp. Có 50,000 hình ảnh trong tập huấn luyện và 10,000 hình ảnh trong tập kiểm tra. Các ảnh trong CIFAR-10 tương ứng với các lớp: Máy bay (Airplane), Xe hơi (Automobile), Chim (Bird), Mèo (Cat), Hươu (Deer), Chó (Dog),Ếch (Frog), Ngựa (Horse), Tàu thủy (Ship), Xe tải (Truck).

**Bảng 3.2** Bảng mô tả dữ liệu tập dữ liệu Cifar-10

| Thuộc tính              | Mô tả                                                                                               |
|-------------------------|-----------------------------------------------------------------------------------------------------|
| Số lượng ảnh            | 60,000                                                                                              |
| Số lớp                  | 10                                                                                                  |
| Kích thước ảnh          | 32x32 pixels                                                                                        |
| Định dạng               | RGB (ảnh màu).                                                                                      |
| Số lượng tập huấn luyện | 50,000                                                                                              |
| Số lượng tập kiểm tra   | 10,000                                                                                              |
| Tên các lớp             | 0: Airplane, 1: Automobile, 2: Bird, 3: Cat, 4: Deer, 5: Dog, 6: Frog, 7: Horse, 8: Ship, 9: Truck. |

Cả hai tập dữ liệu đều có thể tải trực tiếp từ trang web kaggle hoặc thông qua thư viện Tensorflow.

```
(x_train, y_train), (x_test, y_test) = keras.datasets.fashion_mnist.load_data()
```

```
(x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data()
```

Hình 3.4 Tải tập dữ liệu

#### 3.3.1.2.2. Nhóm dữ liệu ứng dụng thực tiễn: PlantVillage (bệnh lá cây) và Chest-Xray-Pneumonia (viêm phổi)

*[Nội dung phần này đang được cập nhật/viết tiếp...]*

#### 3.3.1.3. Các phương pháp Tiền xử lý dữ liệu và Tăng cường dữ liệu (Data Augmentation) cho từng nhóm ảnh

*Chuẩn hóa dữ liệu (Normalization):* Dữ liệu hình ảnh thường được lưu trữ dưới dạng số nguyên từ 0 đến 255, tương ứng với cường độ pixel. Chuẩn hóa dữ liệu chuyển đổi các giá trị này về khoảng [0, 1] bằng cách chia cho

255. Quá trình này giúp các thuật toán học máy hội tụ nhanh hơn và đạt hiệu quả tốt hơn.

*One hot coding:* Quá trình này chuyển đổi các nhãn lớp từ dạng số nguyên sang dạng vector nhị phân. Mỗi nhãn số nguyên được chuyển thành một vector nhị phân có độ dài bằng số lớp (ở đây là 10), với một phần tử là 1 và các phần tử còn lại là 0. Ví dụ, nhãn 3 sẽ được chuyển thành [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]. Giúp các mô hình học sâu (như mạng nơ-ron) có thể xử lý các nhãn này một cách hiệu quả hơn.

#### 3.3.1.4. Hiện thực hóa 3 mô hình đối chứng thực nghiệm:

##### 3.3.1.4.1. Mô hình 1 (SE Baseline): TickNet-small nguyên bản sử dụng chú ý SE trong khối

*[Nội dung phần này đang được cập nhật/viết tiếp...]*

##### 3.3.1.4.2. Mô hình 2 (CBAM-Local): Chèn trực tiếp CBAM thay SE cục bộ trong khối FR-PDP

*[Nội dung phần này đang được cập nhật/viết tiếp...]*



### 3.3.1.4.3. Mô hình 3 (CBAM-Hook - Đề xuất cải tiến): Chèn CBAM phân cấp tại các điểm nối và cuối mạng

[Nội dung phần này đang được cập nhật/viết tiếp...]

#### Xây dựng và cấu trúc mạng CNN thực nghiệm:

**Transfer learning:** là một kỹ thuật trong học máy, trong đó một mô hình được phát triển cho một nhiệm vụ cụ thể được tái sử dụng làm điểm khởi đầu cho một mô hình trên một nhiệm vụ thứ hai. Thay vì bắt đầu từ đầu (từ dữ liệu gốc và mô hình mới), học chuyển giao bắt đầu từ các đặc trưng đã học từ một mô hình đã được huấn luyện trước đó. [12]

**Fine tuning:** là một phương pháp của transfer learning, lấy 1 pre-trained model như VGG, Resnet ... tận dụng 1 phần hoặc toàn bộ các lớp, thêm/sửa/xoá 1 vài lớp để tạo ra 1 model mới. Thường các lớp đầu của model được freeze (đóng băng) lại - tức weight các layer này sẽ không bị thay đổi giá trị trong quá trình train. Lý do bởi các lớp này đã có khả năng trích xuất thông tin mức trừu tượng thấp, khả năng này được học từ quá trình training trước đó. Ta freeze lại để tận dụng được khả năng này và giúp việc train diễn ra nhanh hơn (model chỉ phải cập nhật trọng số ở các lớp cao). [13]

Đối với tập dữ liệu MSNIST-Fashion sẽ được thay đổi kích thước ảnh thành 64x64x3 để phù hợp với việc áp dụng fine tuning vào mô hình. Tải pre-trained model VGG16 từ thư viện Tensorflow và loại bỏ lớp FC. Thêm vào các lớp phù hợp với tập dữ liệu. Đóng băng các lớp đầu của mô hình VGG 16 và hủy đóng băng ở một vài lớp cuối cùng.

Model: "functional\_1"

| Layer (type)                 | Output Shape        | Param #   |
|------------------------------|---------------------|-----------|
| input_layer (InputLayer)     | (None, 64, 64, 3)   | 0         |
| block1_conv1 (Conv2D)        | (None, 64, 64, 64)  | 1,792     |
| block1_conv2 (Conv2D)        | (None, 64, 64, 64)  | 36,928    |
| block1_pool (MaxPooling2D)   | (None, 32, 32, 64)  | 0         |
| block2_conv1 (Conv2D)        | (None, 32, 32, 128) | 73,856    |
| block2_conv2 (Conv2D)        | (None, 32, 32, 128) | 147,584   |
| block2_pool (MaxPooling2D)   | (None, 16, 16, 128) | 0         |
| block3_conv1 (Conv2D)        | (None, 16, 16, 256) | 295,168   |
| block3_conv2 (Conv2D)        | (None, 16, 16, 256) | 590,080   |
| block3_conv3 (Conv2D)        | (None, 16, 16, 256) | 590,080   |
| block3_pool (MaxPooling2D)   | (None, 8, 8, 256)   | 0         |
| block4_conv1 (Conv2D)        | (None, 8, 8, 512)   | 1,180,160 |
| block4_conv2 (Conv2D)        | (None, 8, 8, 512)   | 2,359,808 |
| block4_conv3 (Conv2D)        | (None, 8, 8, 512)   | 2,359,808 |
| block4_pool (MaxPooling2D)   | (None, 4, 4, 512)   | 0         |
| block5_conv1 (Conv2D)        | (None, 4, 4, 512)   | 2,359,808 |
| block5_conv2 (Conv2D)        | (None, 4, 4, 512)   | 2,359,808 |
| block5_conv3 (Conv2D)        | (None, 4, 4, 512)   | 2,359,808 |
| block5_pool (MaxPooling2D)   | (None, 2, 2, 512)   | 0         |
| max_pooling2d (MaxPooling2D) | (None, 1, 1, 512)   | 0         |
| flatten (Flatten)            | (None, 512)         | 0         |
| dense (Dense)                | (None, 1000)        | 513,000   |
| dense_1 (Dense)              | (None, 800)         | 800,800   |
| dense_2 (Dense)              | (None, 400)         | 320,400   |
| dense_3 (Dense)              | (None, 200)         | 80,200    |
| dense_4 (Dense)              | (None, 100)         | 20,100    |
| dense_5 (Dense)              | (None, 10)          | 1,010     |

Total params: 16,450,198 (62.75 MB)  
Trainable params: 4,095,318 (15.62 MB)  
Non-trainable params: 12,354,880 (47.13 MB)

*Hình 3.5 Mô hình CNN cho tập dữ liệu MNIST-Fashion*

Model: "model\_1"

| Layer (type)                             | Output Shape        | Param # |
|------------------------------------------|---------------------|---------|
| input_1 (InputLayer)                     | [(None, 32, 32, 3)] | 0       |
| block1_conv1 (Conv2D)                    | (None, 32, 32, 64)  | 1792    |
| block1_conv2 (Conv2D)                    | (None, 32, 32, 64)  | 36928   |
| block1_pool (MaxPooling2D)               | (None, 16, 16, 64)  | 0       |
| block2_conv1 (Conv2D)                    | (None, 16, 16, 128) | 73856   |
| block2_conv2 (Conv2D)                    | (None, 16, 16, 128) | 147584  |
| block2_pool (MaxPooling2D)               | (None, 8, 8, 128)   | 0       |
| block3_conv1 (Conv2D)                    | (None, 8, 8, 256)   | 295168  |
| block3_conv2 (Conv2D)                    | (None, 8, 8, 256)   | 590080  |
| block3_conv3 (Conv2D)                    | (None, 8, 8, 256)   | 590080  |
| block3_pool (MaxPooling2D)               | (None, 4, 4, 256)   | 0       |
| block4_conv1 (Conv2D)                    | (None, 4, 4, 512)   | 1180160 |
| block4_conv2 (Conv2D)                    | (None, 4, 4, 512)   | 2359808 |
| block4_conv3 (Conv2D)                    | (None, 4, 4, 512)   | 2359808 |
| block4_pool (MaxPooling2D)               | (None, 2, 2, 512)   | 0       |
| block5_conv1 (Conv2D)                    | (None, 2, 2, 512)   | 2359808 |
| block5_conv2 (Conv2D)                    | (None, 2, 2, 512)   | 2359808 |
| block5_conv3 (Conv2D)                    | (None, 2, 2, 512)   | 2359808 |
| block5_pool (MaxPooling2D)               | (None, 1, 1, 512)   | 0       |
| flatten (Flatten)                        | (None, 512)         | 0       |
| dropout (Dropout)                        | (None, 512)         | 0       |
| dense (Dense)                            | (None, 516)         | 264708  |
| dense_1 (Dense)                          | (None, 256)         | 132352  |
| dropout_1 (Dropout)                      | (None, 256)         | 0       |
| dense_2 (Dense)                          | (None, 128)         | 32896   |
| dense_3 (Dense)                          | (None, 64)          | 8256    |
| dropout_2 (Dropout)                      | (None, 64)          | 0       |
| dense_5 (Dense)                          | (None, 10)          | 650     |
| Total params: 15153550 (57.81 MB)        |                     |         |
| Trainable params: 7518286 (28.68 MB)     |                     |         |
| Non-trainable params: 7635264 (29.13 MB) |                     |         |

Hình 3.6 Mô hình CNN cho tập dữ liệu *Cifar-10*

*Đối với tập dữ liệu MNIST-Fashion:* ngoài pre-trained model VGG16 sẽ thêm một lớp maxpool với cửa sổ pooling có kích thước là 2x2 và stride là (2,2). Và thêm lớp Flatten để làm phẳng, tiếp theo là một lớp Dropout với tỉ lệ là 0.5. Tức là các lớp fully connected với số nơ-ron cho các lớp lần lượt là 1000,800,400,200,100 với hàm kích hoạt là ReLU. Cuối cùng là lớp đầu ra với 10 nơ-ron tương ứng với số nhãn của tập dữ liệu với một hàm kích hoạt softmax để phân bố xác suất.

*Đối với tập dữ liệu *Cifar-10*:* ngoài pre-trained model VGG16 sẽ thêm lớp Flatten để làm phẳng, tiếp theo là một lớp Dropout với tỉ lệ là 0.5. Tức là các lớp fully connected với số nơ-ron cho các lớp lần lượt là 516, 256, 128, 64 với hàm kích hoạt là ReLU và 2 lớp Dropout với tỉ lệ 0.5. Cuối cùng là lớp đầu ra với 10 nơ-ron tương ứng với số nhãn của tập dữ liệu với một hàm kích hoạt softmax để phân bố xác suất.

### Áp dụng cơ chế chú ý (Attention Mechanism) vào mô hình CNN:

*Đối với tập dữ liệu MNIST-Fashion:* Một cơ chế CBAM sẽ được thêm vào sau lớp Convolution cuối cùng (block5\_conv3) của pre-trained model Với tỉ lệ giảm  $r = 12$ .

*Đối với tập dữ liệu *Cifar-10*:* Cơ chế CBAM sẽ được thêm vào Convolution cuối của block3, block4 và block5 (block3\_conv3, block4\_conv3, block5\_conv3) của pre-trained model cùng với hàm kích hoạt phi tuyến ReLU và các lớp BatchNormalization. Với tỉ lệ giảm  $r = 2$ .

### 3.3.1.5. Cấu hình quá trình huấn luyện và tối ưu hóa:

#### 3.3.1.5.1. Bộ tối ưu SGD và các hàm Loss tương ứng

Cả 2 tập dữ liệu cho mô hình CNN đều được huấn luyện với hàm loss là “categorical\_crossentropy”, bộ tối ưu hóa là SGD, metric là “accuracy”, Validation\_split là 0.1 với 10 epochs và batch\_size là 32.

#### 3.3.1.5.2. Hàm gọi lại tự động giảm tốc độ học (ReduceLROnPlateau) và dừng sớm (EarlyStopping)

[Nội dung phần này đang được cập nhật/viết tiếp...]

### 3.3.2. Kết quả thực nghiệm, phân tích và đánh giá:

#### 3.3.2.1. Các độ đo hiệu năng chuẩn mực (Accuracy, Precision, Recall, F1-score) và cấu trúc Confusion Matrix

Confusion matrix là một công cụ đánh giá hiệu suất của các mô hình phân loại trong học máy. Nó cung cấp một cái nhìn chi tiết về cách mà mô hình phân loại các

mẫu trong tập dữ liệu kiểm tra (hoặc tập dữ liệu kiểm chứng). Ma trận này chứa các giá trị tổng hợp cho các dự đoán đúng và sai của mô hình, được phân loại dựa trên các lớp thực tế và dự đoán.

|                  |              | Actual Values |              |
|------------------|--------------|---------------|--------------|
|                  |              | Positive (1)  | Negative (0) |
| Predicted Values | Positive (1) | TP            | FP           |
|                  | Negative (0) | FN            | TN           |

Hình 3.7 Cấu trúc Confusion matrix

Trong đó:

- True positive (TP): các điểm dữ liệu dự đoán là 1 và có nhãn là 1.
- True negative (TN): các điểm dữ liệu dự đoán là 0 và có nhãn là 0.
- False positive (FP): các điểm dữ liệu dự đoán là 1 nhưng có nhãn là 0.
- False negative (FN): các điểm dữ liệu dự đoán là 0 nhưng có nhãn là 1.

Confusion Matrix cung cấp thông tin quan trọng để tính toán các chỉ số đánh giá hiệu suất của mô hình phân loại như:

- Accuracy: Cách đơn giản nhất để đánh giá một mô hình phân lớp đó là sử dụng độ chính xác (Accuracy). Ý tưởng đơn giản là tỷ lệ giữa các mẫu dự đoán đúng trên tổng số mẫu của dữ liệu kiểm thử.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

- Precision: Đo lường tỷ lệ các dự đoán tích cực thực sự trong số tất cả các dự đoán tích cực do mô hình đưa ra.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

TP+F

- Recall: Đo lường tỷ lệ dự đoán tích cực thực sự trong số tất cả các trường hợp tích cực thực tế.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

TP+FN

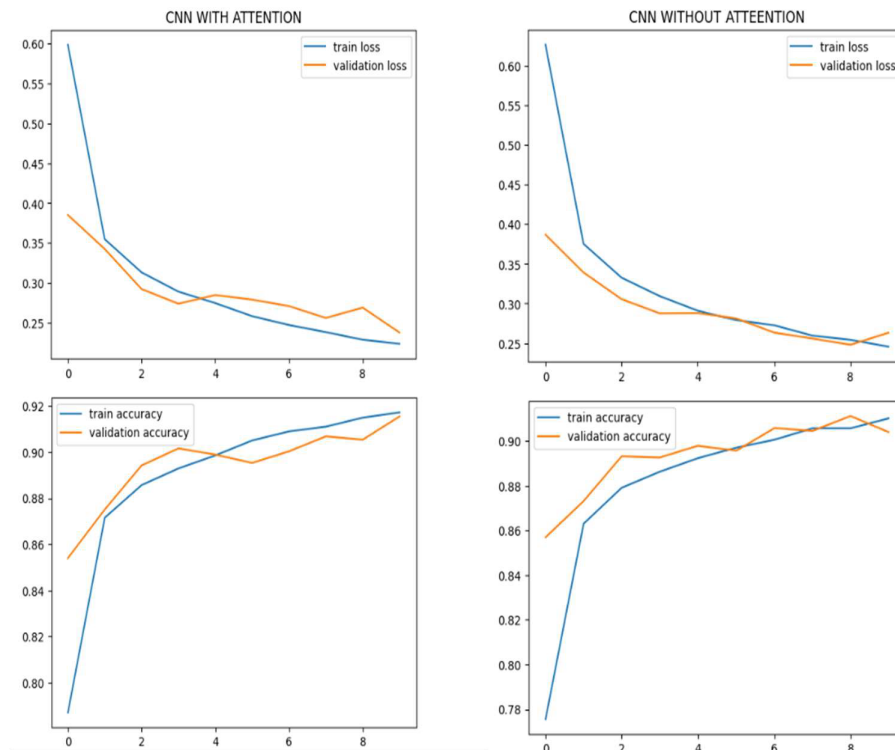
- F1 Score: F1 Score là giá trị trung bình của precision và recall. F1-score có giá trị nằm trong nửa khoảng (0, 1]. F1-score càng cao đồng nghĩa với bộ phân lớp càng tốt. Khi cả recall và precision đều bằng 1 thì F1-score = 1. Khi cả recall và precision đều thấp thì F1-score tiến về 0.

$$\text{F1 Score} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

precision+recall

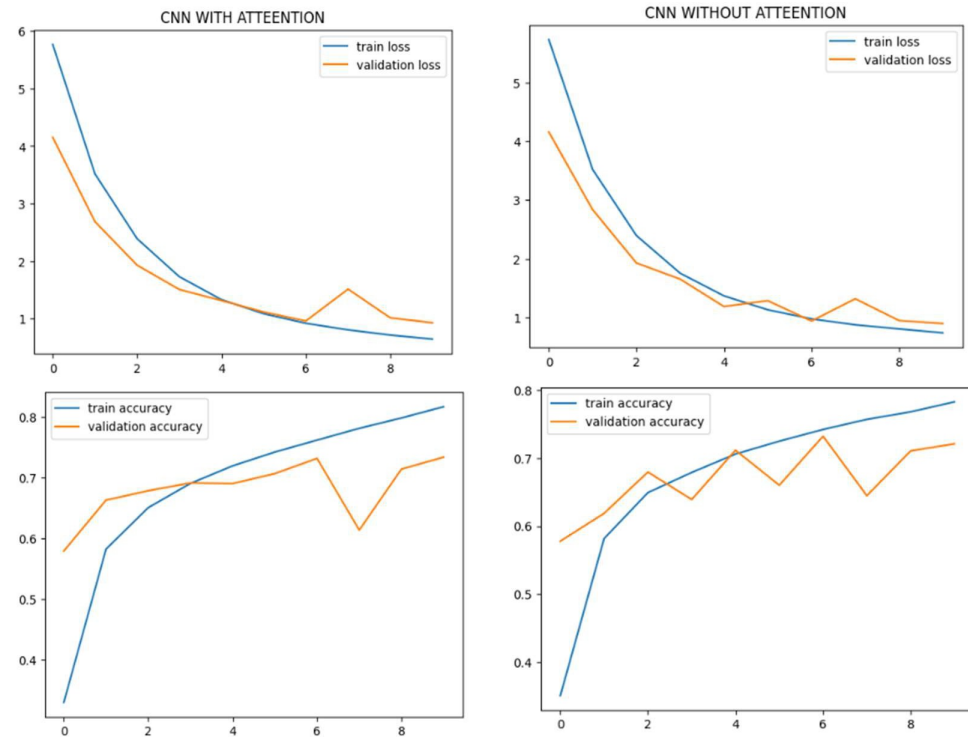
Để đánh giá hiệu quả của các mô hình học viên sẽ dựa vào các chỉ số như accuracy, precision, recall và F1 score trên cả hai tập dữ liệu.

### 3.3.2.2. Kết quả thực nghiệm và biểu đồ Loss/Accuracy trên tập dữ liệu chuẩn (Fashion-MNIST, CIFAR-10)



Hình 3.8 Biểu đồ loss và accuracy của mô hình CNN có áp dụng Attention và không áp dụng với tập dữ liệu MNIST-Fashion

Nhìn chung cả hai mô hình đều có xu hướng giảm loss tương tự nhau. Training loss và validation loss của cả hai mô hình đều giảm dần và giữ xu hướng giảm gần như song song sau khoảng epoch 2. Mặc dù cả hai mô hình có xu hướng rất giống nhau, nhưng mô hình với Attention có accuracy ổn định và cao hơn một chút so với mô hình không có Attention.



Hình 3.9 Biểu đồ loss và accuracy của mô hình CNN có áp dụng Attention và không áp dụng với tập dữ liệu Cifar-10

Cả hai mô hình đều cho thấy giảm training và validation loss qua các epoch. Mô hình có attention dường như có đường cong validation loss mượt hơn một chút, cho thấy quá trình học ổn định hơn. Về Accuracy thì mô hình có attention có sự tăng trưởng training accuracy ổn định hơn. Validation accuracy của mô hình có attention cũng ổn định hơn so với mô hình không có attention, mô hình không có attention có những dao động rõ rệt hơn. Qua đó có thể thấy việc thêm các cơ chế attention vào CNN dường như giúp ổn định quá trình huấn luyện và có thể cải thiện khả năng của mô hình trong



việc tổng quát hóa với dữ liệu chưa thấy trước đó.

**Bảng 3.3** Bảng so sánh kết quả giữa các giải thuật trên tập dữ liệu MNIST-Fashion

| Algorithm             | Accuracy      | Precision    | Recall        | F1            |
|-----------------------|---------------|--------------|---------------|---------------|
| KNN                   | 0.8389        | 0.8727       | 0.8389        | 0.8536        |
| RandomForest          | 0.734         | 0.734        | 0.734         | 0.7047        |
| DecisionTree          | 0.7603        | 0.7648       | 0.7603        | 0.7603        |
| CNN without Attention | 0.8984        | 0.9001       | 0.8984        | 0.8959        |
| CNN with Attention    | <b>0.9066</b> | <b>0.907</b> | <b>0.9066</b> | <b>0.9066</b> |

**Bảng 3.4** Bảng so sánh kết quả giữa các giải thuật trên tập dữ liệu Cifar-10

| Algorithm             | Accuracy      | Precision    | Recall        | F1            |
|-----------------------|---------------|--------------|---------------|---------------|
| KNN                   | 0.3567        | 0.4385       | 0.3567        | 0.3466        |
| RandomForest          | 0.4356        | 0.4356       | 0.4356        | 0.4267        |
| DecisionTree          | 0.311         | 0.3101       | 0.311         | 0.311         |
| CNN without Attention | 0.7157        | 0.7282       | 0.7157        | 0.714         |
| CNN with Attention    | <b>0.7265</b> | <b>0.737</b> | <b>0.7265</b> | <b>0.7292</b> |

Nhìn vào bảng 3.3 và 3.4 có thể thấy được mô hình CNN có áp dụng cơ chế attention CBAM mang lại hiệu suất tốt hơn so với các mô hình không áp dụng với các chỉ số như accuracy, precision, recall và f1 score. Trong cả hai bảng mô hình CNN áp dụng cơ chế attention đều mang lại hiệu quả tốt hơn so với các mô hình không áp dụng. Đặc biệt trong bảng 3.4 các phương pháp học truyền thống như KNN, RandomForest và DecsionTree cho thấy hiệu suất không ổn định và thấp, điều này chỉ ra rằng phương pháp học truyền thống không đủ mạnh để giải quyết các bài toán phân loại hình ảnh yêu cầu độ phức tạp cao. Ngược lại trong cả 2 bảng mô hình CNN đều mang lại hiệu quả khá tốt, đặc biệt là khi áp dụng cơ chế attention, mô hình CNN có áp dụng cơ chế attention đều mang lại accuracy, precision, recall và f1 score cao hơn so với các phương pháp còn lại. Điều này cho thấy cơ chế attention có tác động tích cực đến hiệu suất mô hình. Có thể thấy được CBAM giúp mô hình tập trung vào các đặc trưng quan trọng trong hình ảnh. Cơ chế Channel Attention làm nổi bật các

kênh chứa thông tin quan trọng, trong khi Spatial Attention giúp mô hình tập trung vào các vùng không gian quan trọng trong hình ảnh. Sự kết hợp này giúp cải thiện hiệu suất của mô hình một cách toàn diện. Qua đó có thể chứng minh rằng học sâu kết hợp với cơ

chế chú ý có thể cung cấp hiệu suất phân loại tốt hơn đáng kể so với các phương pháp truyền thống.

### **3.3.2.3. Kết quả thực nghiệm và biểu đồ Loss/Accuracy trên tập dữ liệu thực tiễn (PlantVillage, Chest-Xray-Pneumonia)**

*[Nội dung phần này đang được cập nhật/viết tiếp...]*

### **3.3.2.4. Phân tích hiện tượng suy giảm hiệu năng cục bộ (CBAM-Local) và ưu thế của giải pháp đề xuất (CBAM-Hook)**

*[Nội dung phần này đang được cập nhật/viết tiếp...]*

### **3.3.2.5. Giải thích mô hình bằng trực quan hóa bản đồ nhiệt Grad-CAM (Sự tập trung vùng không gian đặc trưng thực tế)**

*[Nội dung phần này đang được cập nhật/viết tiếp...]*

## **CHƯƠNG 4: KẾT QUẢ, KẾT LUẬN VÀ KIẾN NGHỊ**

### **4.1. KẾT QUẢ ĐẠT ĐƯỢC CỦA ĐỀ TÀI**

Trong bài tiểu luận này, thông qua các tài liệu về mô hình mạng nơ-ron tích chập (CNN) và cơ chế chú ý CBAM, học viên đã hiểu kiến trúc, cách thức hoạt động của mô hình CNN, cùng với đó là cơ chế chú ý và áp dụng vào mô hình CNN. Học viên đã khảo sát và hoàn thành thực hiện nghiên cứu việc áp dụng cơ chế chú ý CBAM vào mô hình CNN với pre-trained model VGG16 trên mặt lý thuyết lẫn thực nghiệm trên hai tập dữ liệu MNIST-Fashion và Cifar-10. Học viên đã xây dựng được cơ chế chú ý CBAM áp dụng vào sau một vài lớp của mô hình CNN và thực hiện huấn luyện cùng với một số mô hình không áp dụng. Từ các số liệu từ hình 3.8, 3.9 và bảng 3.3, 3.4 so sánh accuracy, precision, recall và f1 score thu được thông qua thực nghiệm đã cho thấy được mô hình CNN có áp dụng cơ chế chú ý CBAM đã mang lại kết quả tích cực cho mô hình.

### **4.2. KẾT LUẬN (Đánh giá mức độ hoàn thành so với mục tiêu và ý nghĩa thực tiễn)**

Qua những thông số thu được từ kết quả thực nghiệm cho thấy kết quả của mô hình CNN thật sự tốt hơn so với các mô hình máy học truyền thống, đặc biệt là việc áp dụng cơ chế chú ý CBAM vào mô hình đã cải thiện hiệu suất cho mô hình thông qua các chỉ số như accuracy, precision, recall và f1 score. Việc tích hợp CBAM vào mô

hình CNN đã chứng minh được hiệu quả trong việc nâng cao độ chính xác và hiệu suất tổng thể của mô hình phân loại hình ảnh. Cơ chế Attention giúp mô hình tập trung vào các đặc trưng và vùng quan trọng, cải thiện khả năng tổng quát hóa. Những kết quả này khẳng định rằng việc sử dụng cơ chế chú ý là một hướng đi triển vọng trong việc phát triển các mô hình học sâu cho bài toán phân loại hình ảnh.

#### 4.3. KIẾN NGHỊ VÀ CÁC HƯỚNG PHÁT TRIỂN TIẾP THEO

Cơ chế chú ý đã thật sự mang lại hiệu quả tốt cho mô hình CNN, vì vậy thông qua nghiên cứu này có thể nghiên cứu thêm các cơ chế chú ý khác và áp dụng nhiều cơ chế chú ý khác nhau để tận dụng các ưu điểm của từng loại để đạt được hiệu suất cao hơn. Nghiên cứu các phương pháp tối ưu hóa để giảm thiểu chi phí tính toán của cơ chế chú ý mà vẫn giữ được hiệu suất cao. Nghiên cứu mở rộng việc áp dụng cơ chế chú ý vào các nhiệm vụ và các mô hình học sâu.

#### TÀI LIỆU THAM KHẢO

1. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin “Attention is all you need”, 2017.
2. Trang web [What is Convolutial Neural Network? Begin’s Tutorial for Machine Learning and Deep Learning](#), truy cập ngày 12/03/2024.
3. Altenberger, Felix, and Claus Lenz. “A non-technical survey on deep convolutional neural network architectures.” arXiv preprint arXiv:1803.02129 (2018).
4. Cao, Changyu et al. “A convolutional neural network face recognition algorithm based on data augmentation.” (2019)
5. Simonyan, K.; Zisserman, A. Very deep convolutional networks for large-scale image recognition. arXiv 2014, arXiv:1409.1556..
6. Trang web [Thuật toán CNN là gì? Và cấu trúc mạng Convolution Neural Network](#), truy cập ngày 5/4/2024.
7. Guo, M.-H., Xu, T.-X., Liu, J.-J., Liu, Z.-N., Jiang, P.-T., Mu, T.-J., ... & Wang, Y. (2022). Attention mechanisms in computer vision: A survey. Computational Visual Media, 8, 331-368.

8. Trang web Attention Mechanisms in Computer Vision: CBAM, truy cập ngày 10/05/2024.
9. Sanghyun Woo, Jongchan Park, Joon-Young Lee, In So Kweon “CBAM: Convolution Block Attention Module”.
10. Jie Hu, Li Shen, Samuel Albanie, Gang Sun, Enhua Wu “Squeeze-and-Excitation Networks”.
11. Trang web CBAM-keras, truy cập ngày 15/05/2024.
12. Trang web Understanding Transfer Learning for Deep Learning, truy cập ngày 15/05/2024.
13. Trang web Fine tuning pre-trained model trong pytorch và áp dụng vào Visual Saliency Prediction, truy cập ngày 16/05/2024.